

Università degli Studi di Trento

Department of Industrial Engineering

Master's Degree in Management and Industrial Systems Engineering

Route Consistency in the Periodic Vehicle Routing Problem: A Multi-Objective Approach for Waste Collection

Supervisor
Prof. Francesco Pilati

Student
Camilla Menon
249822

Academic Year 2024/2025

A chi crede
anche nelle notti più buie.
A chi ha il coraggio
di mostrare lo splendore.

Acknowledgments

I would like to thank Prof. Francesco Pilati (University of Trento) and Prof. Mag. Dr. Karl Franz Dörner (University of Vienna) for their guidance and valuable feedback throughout this work.

A special thanks goes to PhD researcher Alina-Gabriela Müller (formerly Dragomir, University of Vienna) for her constant support, encouragement, and practical help, which have been essential for the completion of this thesis.

Their support has been fundamental, and I am truly grateful for the opportunity to work under their supervision.

Abstract

The study addresses the Periodic Vehicle Routing Problem (PVRP) for waste collection, considering fluctuations in demand. The main objective is to analyze the trade-off between minimizing distance traveled and maximizing route consistency, which is crucial for operational efficiency and driver familiarity. Multi-objective modeling combines distance with three similarity metrics (*Nodes-set*, *Arc-sequences*, *Arcs-set*), and optimization was conducted using the Adaptive Variable Neighborhood Search (A-VNS) algorithm. The results show that A-VNS effectively balances competing objectives, highlighting intrinsic trade-offs between different texture shapes and guiding perturbed routes to reference-like configurations.

Contents

Acknowledgments **iii**

Introduction **1**

1 Optimization of Vehicle Routing for Waste Collection **5**

1.1 Waste collection 5

1.1.1 Decision-making factors 6

1.1.2 Fluctuation of municipal waste 7

1.1.3 Scope and objectives 8

1.1.4 Literature review on Waste Collection 8

1.2 Vehicle routing problems 9

1.2.1 Period Vehicle Routing Problem 10

1.2.2 Route consistency in vehicle routing 11

1.2.3 Multi-objective function 13

1.2.4 Optimization algorithms 13

1.2.5 Literature review on optimization algorithms for PVRP 16

2 Problem description **19**

2.1 Operational context of the problem 19

2.1.1 Problem Workflow 20
2.2 Reference and perturbed instances 20
2.2.1 Reference instances 20
2.2.2 Demand fluctuation for perturbed instances 21
2.3 Conceptual consistency metrics 23
2.3.1 Definition of the aggregated objective function 23

3 Mathematical formulation **25**

3.1 Period Vehicle Routing Problem formulation 25
3.2 Demand fluctuation modeling 28
3.3 Formalization of consistency metrics and aggregated objective 29
3.3.1 Nodes-set consistency 30
3.3.2 Arc-sequences consistency 31
3.3.3 Arcs-set consistency 32
3.3.4 Weighted objective function 33

4 Solving approaches **35**

4.1 Primary solution 35
4.1.1 Assignment Algorithm 35
4.1.2 Completion Algorithm 37
4.1.3 VRP Optimization Algorithm 38
4.2 Refined solution 39
4.2.1 Variable Neighborhood Descent Algorithm 39
4.2.2 Neighborhoods 41
4.2.3 Adaptive Variable Neighborhood Search Algorithm 46

4.3 Summary: approach to the complete problem 49

4.3.1 Experiments and implementation details 51

5 Results **53**

5.1 Primary solution results 54

5.2 Neighborhood efficiency results 57

5.3 Multi-objective algorithm results 63

5.3.1 Evaluation of objective improvement 63

5.3.2 Correlation between consistency measures 68

5.3.3 Instance routes analysis 72

Conclusions **75**

References **81**

Appendix A **83**

Appendix B **89**

Introduction

Solid Waste Management (SWM) represents an essential public utility for the functioning of contemporary society. According to the Organisation for Economic Co-operation and Development (OECD), it includes all operations dedicated to the collection, transport, treatment, and disposal of Municipal Solid Waste (MSW), as well as activities to regulate and prevent waste generation through process modification, reuse, and recycling [1]. Within this system, the collection phase (Waste Collection, WC), understood as the collection and transfer of waste to treatment plants by public or private services, plays a fundamental role in maintaining adequate conditions of health and urban quality.

The importance of waste collection has roots in modern urban history: as early as the 19th century, the first European regulations aimed to contain epidemics such as cholera [2]. Today, the motivations are broader and more complex. Population growth and urbanization have in fact significantly increased waste generation, which in 2020 reached 2.24 billion metric tons globally, with a forecast of 3.88 billion by 2050 [3]. This makes waste transport management a crucial economic, environmental and social challenge. Waste collection is one of the highest-cost phases in the entire supply chain, accounting for 60% to '80% of overall treatment costs [4]. Added to this are the environmental impacts of transport, which are responsible for a significant contribution to global greenhouse gas emissions and fuel consumption. An inefficient collection system also has negative effects on the well-being of workers and citizens: irregular routes, frequent rescheduling, and the discontinuous allocation of operational areas lead to increased health risks, lower service quality, and a general deterioration in operational efficiency [3].

In this complex context, the problem of planning vehicles dedicated to collection (Waste Collection Vehicle Routing Problem, WCVRP) arises. In addition to the traditional objectives of minimizing costs and distances traveled, recent years have seen a growing focus on operational aspects more closely aligned with the daily reality of collection services. Among these, an increasingly important role is assumed by the concept of route consistency, that is, maintaining

a high degree of consistency in vehicle and route assignments over time. Consistency responds to concrete operational needs: drivers and operators develop familiarity with their service areas, reducing driving and collection times, and increasing service reliability. Less variability in assignments also contributes to organizational stability, promotes better workload distribution, and reduces the risk of errors, especially in complex urban contexts. This objective takes on a relatively important value, especially in response to foreseeable and unforeseeable events, which significantly influence the rate of waste production. Seasonality, holidays, but also unexpected events that require the interruption of collection services, such as heavy snowfall, pose a challenge in WCVRP. Simply re-optimizing routes based on costs or total distance traveled may initially seem like the most effective solution for this type of event. What is not considered is that such a sudden change in the daily work of operators represents not only a factor of less satisfaction, but also a longer time to complete the task. The operators themselves have in fact declared that knowledge of the location of the bins and greater familiarity with the routes, allows them to carry out the work more easily and in less time. In light of this, the consistency of the routes emerges as a complementary but essential objective alongside traditional efficiency. Considering these observations, it is possible to affirm that the consistency of the routes emerges as a complementary but essential objective alongside traditional efficiency.

This study aims to analyze the influence of *consistency* on the distances associated with routes, comparing standard route programming with perturbed solutions, where waste rates have fluctuated. The similarity measures considered are the *Nodes-set consistency*, the *Arc-sequences consistency* and the *Arcs-set consistency*. These quantities are integrated with the total distance measure of the solution within a weighted objective function. To evaluate its effect, several scenarios were defined, represented by different weights assigned to distance and consistency: $100 - 0$, i.e. exclusive distance optimization, $75 - 25$, $50 - 50$, $25 - 75$, and finally $0 - 100$, corresponding to exclusive consistency optimization.

The first chapter introduces the context of *Waste Collection*, analyzing decision factors, waste fluctuation, service objectives and goals, and presents the Vehicle Routing Problem (VRP), with a focus on the Periodic VRP (PVRP), which forms the basis of waste collection problems. The study goes into detail starting from the second chapter, where the main characteristics of the problem under analysis are described. Next, the mathematical formulation of the PVRP and the distribution used to model customer requests are presented, along with definitions of similarity measures and the weighted objective function. The fourth chapter illustrates all the algorithms and solution approaches used to determine the initial solutions and their optimized versions, both based on distance alone and in combination with similarity measures. Finally, the fifth chapter focuses on the in-depth analysis of *consistency*, evaluating its impact on the distances traveled and the overall performance of the solutions. This chapter aims to highlight how integrating similarity measures can guide route optimization, improving solution quality and assignment

stability even in the presence of fluctuations in customer demand.

Chapter 1

Optimization of Vehicle Routing for Waste Collection

1.1 Waste collection

Waste Management (WM) has been defined by Organisation for Economic Co-operation and Development (OECD) as the system responsible for the collection, transportation, treatment, and disposal of Municipal Solid Waste (MSW), as well as for its regulation and prevention through process modification, reuse, and recycling [1]. A relevant operation in WM is Waste Collection (WC), which is "is the collection and transport of waste to the place of treatment or discharge by municipal services or similar institutions, or by public or private corporations, specialized enterprises or general government" [1]. This process is responsible for retrieving waste from public or private garbage cans and transporting it to the disposal facility. Based on the selected disposal method, the disposal facility could be a recycling plant, an incinerator for energy recovery, or a landfill [5]. Furthermore, collection can be undifferentiated, meaning all types of waste are mixed, or differentiated, which requires collecting separately different types of waste [1]. By combining the disposal method with the waste sorting approach, a variety of differentiated management paths emerge, each corresponding to specific types of waste. This diversification increases the complexity of collection and transport operations, as each waste stream may require separate handling procedures and dedicated vehicles.

Waste collection is a specific problem of the Vehicle Routing Problem (WCVRP) [5]. VRP was first introduced in 1959 with the aim of optimizing routes for forward deliveries, which are

transportation from a warehouse to "n" customers [6]. In WCVRP, numerous collection points must be served to send waste to a single, in some exceptions multiple, destination facility. This is a reverse logistics problem, where many pickup points converge to a single delivery point [5]. The WCVRP typically involves a fleet of vehicles that must visit collection points (stops) or bins over a longer period or day. Routes typically begin and end at the depot, but after collection and before route closure, vehicles must visit the disposal facilities. In WCVRP, the goal is to determine the optimal route used by a group of vehicles to serve a group of users, meeting customer requirements and minimizing the overall cost of transportation, fulfilling all operational constraints [5].

1.1.1 Decision-making factors

The decision-making factors and aspects that make the waste collection problem (WCVRP) complex extend on several levels: strategic, tactical, and operational [7]. The design of a waste collection system requires a series of interconnected decisions that concern different operational and strategic aspects, which are medium to long-term decisions. Starting with the location of infrastructure, such as collection centers, vehicle depots, and treatment or disposal facilities, often referred to as Intermediate Facilities (IFs). The choice of such locations influences the division of the territory into operational zones, each manageable by a single vehicle [7]. Another crucial element is the composition and type of containers, which, as mentioned, depends on the type of waste classification selected. The type of collection system must also be established, choosing between door-to-door mode, the bring-system, in which citizens deliver waste to centralized locations, or hybrid solutions that combine both methods [7]. At the same time, it is necessary to determine the most appropriate modes of transport by selecting the collecting vehicles, which can be classified into single-compartment trucks and multi-compartment trucks.
Operational decision-making factors concern the day-to-day management of waste collection operations and form the core of the WCVRP [7]. The primary goal is to establish the best routes for the fleet to efficiently serve all collection and unloading points. Vehicle capacity and flexibility constraints are also included: every vehicle has capacity limitations and can be part of a diverse fleet, which can contain vehicles with varying technologies or compartments. Additionally, vehicles often have to make multiple trips to the disposal facility to empty the cargo before resuming collection [5]. Flexibility constraints refer to the heterogeneity of the fleet, meaning that vehicles may differ in capacity, technology (e.g., rear-loading vs side-loading trucks), or compartment configuration. These differences limit the interchangeability of vehicles across routes and require assigning customers to vehicles compatible with their waste type and loading system. A further operational element is represented by time constraints, which require compliance with time slots for service, staff breaks and the maximum duration of work shifts. The nature of the demand also introduces complexity: container filling levels are difficult to predict

and can be treated as stochastic variables, especially in the absence of automatic monitoring systems, and the presence of different types of waste requires collection protocols and dedicated means. Finally, the context of urban logistics is a factor that can further complicate the problem with the introduction of traffic conditions, one-way road constraints, which must be integrated into planning to avoid congestion and ensure efficient operations.

1.1.2 Fluctuation of municipal waste

Waste Collection demand is determined by the rate of material accumulation at a specific collection point, container, or bin at a given time. This is represented by the amount of waste that must be gathered. Historically, Waste Collection management has been based on static, predefined routes, performed regularly [8]. In this approach, parameters are often assumed to be deterministic, that is, known or constant. The demand for collection is actually non-deterministic, which introduces a major structural change to the problem [9]. To handle the uncertainty and dynamics of the volumes produced, in waste collection problems the accumulation rate is often modeled as a stochastic variable.
The generation and composition of waste, and therefore the demand for collection, vary significantly over time and as a function of geographical area [10]. The nature of events that affect the demand can determine whether they have a short- or long-term impact. On the day-to-day changes scale, several studies highlight an increase in waste generation and disposal on weekends compared to weekdays [10]. This happening is due in part to commercial catering activities, which during the weekend observe a greater presence of customers and therefore an increase in waste production. Waste generation and disposal show seasonal variations, with levels generally higher in summer months than in the winter period [10]. On larger time scales, collection demand is influenced by structural and strategic factors. Socio-economic growth and urbanization tend to increase the total amount of waste produced, while demographic and behavioral changes can reduce it and promote recycling [10]. Long-term policies, such as recycling targets and management plans, also define future scenarios that impact service planning [11]. Furthermore, specific events can significantly influence collection demand. Holidays, seasonal demonstrations, and temporary population movements —such as the arrival or departure of students— generate spikes in waste [12]. In some periods, increases related to particular activities, such as spring cleaning or agricultural harvesting, are observed, which affect the quantity and type of materials produced [12]. Furthermore, in tourist areas the flow of visitors significantly changes both the volume and the composition of waste. Another factor must be taken into account, which does not directly affect the rate of waste generation but rather in terms of operational interference. Atmospheric phenomena, such as snowfall, can disrupt scheduled collections [10]. As a result, a greater amount of waste accumulates in the following days to be disposed of, as the service must recover both the waste from the day of the outage and that produced in the following days.

Heavy rainfall can affect the quantity and composition of waste, as water can impregnate waste before collection or accumulate in vehicles, increasing the load to be handled [10].
For the reasons mentioned above, it is clear to understand that a deterministic approach, based on fixed-routes or that relies on the expected value of demand, is limited. The use of static routes does not take into account the actual filling level of bins, leading to high resource inefficiencies, due to too early collection of unfilled bins or poor service for too late collection. These observations highlight the need in the waste collection sector to adopt dynamic models or recalculate routes in response to unexpected events.

1.1.3 Scope and objectives

The primary goal of WCVRP has traditionally been to minimize the total cost of the route, but the focus has expanded to include environmental and social objectives, often through multi-objective formulations. Cost in VRP is defined as the sum of fixed vehicle costs, representing rental cost or operating costs, and variable costs [13]. The variable expenditures can be measured as a function of the distance traveled or the time taken, and represent the vehicle's consumption. Additionally, penalties, or additional costs, can be added to the cost function to enforce certain constraints on vehicle limits [13]. The other most common objectives of routing problems are minimizing travel distance or time, accompanied by maximizing profit [7]. Other objectives focus on strategic factors such as minimizing the number of vehicles to reduce the fleet required for the collection system, or optimizing placement, which aims to find the most appropriate location for collection bins and disposal facilities.
Many studies address Waste Collection problems with two, or in some cases more, objectives, thus dealing with secondary aspects in addition to cost minimization. Among the secondary problems, attention to social and quality of service objectives are other issues present in the context of WCVRP. These objectives aim to improve operators' working conditions and the public's perception of the service. Workload balancing aims to distribute work equally among vehicles or operators. Public interest objectives include maximizing citizen satisfaction while minimizing public inconveniences, defined as "unpleasant effects" such as discomfort resulting from the placement of systems or odor control, which is particularly relevant in areas characterized by hot climates [7]. These social aspects are closely related to route consistency, since drivers benefit from serving familiar areas and maintaining stable route assignments across days

1.1.4 Literature review on Waste Collection

The Waste Collection Vehicle Routing Problem is widely studied in the literature, with multiple papers exploring different objective functions and specific constraints, in addition to classical cost or distance minimization. Starting in 1974 with the publication of Beltrami and Bodin [14],

which introduced the Period Vehicle Routing Problem (PVRP) for planning municipal waste collection, many other researchers have studied the topic. Torres and Antón [15], Apaydin and Gonullu [16], and Ghose et al. with a study on Asansol municipality [17] worked on distance minimization in the context of municipal waste collection. Alagöz and Kocasoy [18] and Kim et al. [19] have optimized the distance traveled of special waste collection systems, specifically on health-care wastes the former and on end-of-life recycling consumer electronic goods the latter. Authors such as Clark and Gillean [20], and Arribas et al. [21] focus on optimizing resource use, particularly the vehicle fleet. Part of the Waste Collection research is dedicated to infrastructure optimization through analysis of routing costs combined with the costs of installing and locating delivery stations. This type of problem is called Location Routing Problems (LRPs) and has been addressed by Chang and Wei [22], Hemmelmayr et al. [23], and Cubillos and Wøhlk [24], which aims to maximize area coverage and, implicitly, the amount of waste recycled. More current issues, such as sustainability and thus minimizing environmental impact in WC, have been discussed by Bing et al. [25], Xin et al. [26], and Ramos et al. [27], who respectively considered eco-efficiency and fuel consumption minimization aspects, for the latter two.

1.2 Vehicle routing problems

The Vehicle Routing Problem (VRP) is an optimization problem that was first proposed by Dantzig and Ramser in 1959 [6]. Initially, the problem was formulated to model how a fleet of homogeneous trucks could meet the demand for oil from a number of gas stations starting from a central hub, trying to minimize the total distance traveled [13]. Classical VRP is based on several fundamental assumptions. First of all, each customer, represented by a node in the graph, is served exactly once by only one vehicle, and each customer's demand is indivisible, so fractional delivery is not allowed. Each vehicle starts and ends its route from a depot and may be subject to constraints such as vehicle capacity, time slots, or other operational limitations. The problem is to find optimal routes for a fleet of vehicles to serve a group of customers. The closed paths of the solution constitute an optimal subset of edges of the graph [6]. In theoretical formulations of VRP, it is often assumed, implicitly or explicitly, that the graph is complete [14]. However, in practice, the graph does not require completeness. The graph represents the existing road network and the shortest distances. Routing problems on nodes (Node Routing Problems, NRP) are typically defined with a complete graph, while routing problems on edges (Arc Routing Problems, ARP) are often represented by an incomplete graph [7].

The field of the Vehicle Routing Problem has developed over time to include real-world complexities, giving rise to numerous variants. Each variant introduces additional constraints or features to the VRP, resulting in a specific version or subcategory of the problem. Capacitated VRP

(CVRP) and VRP with Time Windows (VRPTW) are the most common variants. In VRPTW, deliveries or services to customers must take place within predefined time intervals, which determine the time constraints. Among the variants, VRP with time-windows remains one of the most common modeling goals in current research, closely related to customer service satisfaction [13]. The VRP with Heterogeneous Fleets (HFVRP) is characterized by a fleet of vehicles of different types, such as different capacities or costs. In some delivery or pickup scenarios, route planning may involve multiple depots or origin points, as in Multi-Depot VRP (MDVRP). To make the representation of the road network more realistic, it is possible to include variable travel times, for example due to traffic, as in Time-Dependent VRP (TD-VRP). In the waste collection industry, it's also common to allow vehicles to unload or reload at intermediate facilities, such as transfer stations or disposal facilities, thus restoring capacity along the way; this variant is known as VRP with Intermediate Facilities (VRP with IF).
In addition to the specific variants of the VRP just discussed, which introduce new constraints or operational features, there are also generalizations of the problem that extend the model by increasing the variables and decision factors involved. The Location Routing Problem (LRP) combines strategic decision-making about the location of depots, treatment centers, or bins with operational decision-making about the route. Problems that integrate routing decisions with inventory management decisions, such as when and how much to collect or deliver goods, to prevent depletion or, as in waste collection, to cite overflows, are called Inventory Routing Problems (IRP). In the case of parameter uncertainty, such as customer demand, travel times, or customer presence, they are modeled as stochastic, and this variant is called Stochastic VRP (SVRP). If the information changes over time while the vehicles are currently on the road, the routes are recalculated in real-time and the VRP changes to a dynamic aspect (Dynamic VRP, DVRP).

1.2.1 Period Vehicle Routing Problem

The periodic vehicle routing problem (PVRP) is an extension of the standard VRP and was introduced in the context of waste collection by Beltrami and Bodin in 1974 [14]. In PVRP vehicle routes are planned over a time horizon, defined as period, of several days. It is important to distinguish between the Periodic VRP (PVRP) and the Multi-period VRP. In the PVRP, customers require a fixed number of visits within the planning horizon and the model decides on which days each customer must be served. In a Multi-period VRP, instead, each day has its own VRP and the demand, availability, or constraints may vary across days, but no visit-frequency decisions are involved. In PVRP, clients request repeat visits over the given time horizon. Each client is distinguished by a frequency, which determines the exact number of times it has to be visited during the temporal arch. Additionally, the visit plan is chosen from a set of possible visits scenario, where each option represents a set of days on which the customer has to be visited. To give an example, for a customer with a frequency of 2 over a period of 5 working days it is

undesirable for visits to take place on consecutive days. Visit options will then be determined by pairs of days spread over the period so as to provide as much coverage as possible. Therefore, visit plans could be: Monday and Wednesday, Tuesday and Thursday, or Wednesday and Friday.

PVRP's goal is to identify a set of routes that both meet demand and frequency requirements, while minimizing the total expenses of all paths over the planning period. Coene and Spieksma defined PVRP as a problem "situated on the border between tactical and operational planning" [28]. Given these characteristics, PVRP is often approached as a two-stage problem, combining visit plan selection (Assignment) with daily route optimization (Routing) [28]. In the assignment phase, customers are assigned to specific days or visit scenarios within the planning horizon based on their service needs. In routing, a VRP problem (or VRP with variants, such as VRPTW) is solved for each individual day in the scheduling horizon, using the customers assigned to that day. A solution approach is the Cluster first, Route second method [28]. The first step involves assigning customers to the days, either seeking a uniform distribution of workload or using a location-based clustering approach. Next we move on to the daily resolution of the VRP. This approach tends to produce non-overlapping regions, which are considered easier to control and manage [14]. The Route first, Cluster second approach, on the other hand, reverses the two phases [28]. The routes built in the first phase are giant tours, which visit all customers. Then, the giant tour is divided or "partitioned" into a series of routes that, with respect to capacity and time constraints, and customer frequency. Finally, as a final step, the VRP for each day of the period is also resolved in this case in order to improve the routes.

1.2.2 Route consistency in vehicle routing

The Period VRP, just like the VRP, has evolved to incorporate real-world complexity, generating numerous variants. In PVRP with Service Choice (PVRP-SC) the service frequency is not a fixed parameter, but a decision variable within the model [29]. Each client requires a minimum frequency of visits during the period. Increased delivery/collection frequency can lead to benefits, such as customer satisfaction, while also requiring higher service costs. The goal is therefore to minimize total travel costs minus the benefit from the service. The PVRP-SC has proven to be not only more flexible—since service frequency becomes a decision variable rather than a fixed input—but also more efficient than the classical PVRP formulation, because it allows reducing unnecessary visits while increasing service quality [29]. The PVRP with Time Windows and Diffuse Time Constraints (PVRP-TWTS) is a variant typically found in the high-value shipping transportation area [30]. Time windows define when the customer is available for the visit. Additionally, due to security concerns, the arrival time at a customer must be irregular between two visits. Therefore, spread time constraints are inserted, which imply a variation in customer

visits of a certain time constant.

The concept of consistency in vehicle logistics and routing (VRP) is fundamental and refers to the provision of a stable and predictable service over time [31]. Consistency is considered the key characteristic of high-quality service and, consequently, high customer satisfaction. Customers appreciate a service provided at regular times and, ideally, by the same driver [31]. Driver consistency improves the driver-customer relationship and enables personalized service. Furthermore, drivers' familiarity with customers, routes and traffic conditions is exploited, reducing the learning burden [31]. For drivers, consistency decreases the learning load and improves their productivity. Finally, consistency reduces the effort required to readjust routing plans on a daily basis [31].
Service consistency can be measured mainly by three dimensions: Arrival Time Consistency, Person-Oriented Consistency, and Delivery Consistency. Ensuring consistency in time means that different visits to the same customer take place, from day to day, at more or less the same time [32]. This aspect is particularly important in services such as home health care, for example for the administration of drugs. Time consistency can be modeled as a hard constraint, imposing a maximum allowable variation in arrival time, or as a soft constraint, penalizing any deviations between visiting hours in the objective function [31]. Person-oriented consistency concerns the continuity of service from the customer and driver's perspective. Driver consistency refers to each customer being visited by the same operator or vehicle each time, ensuring continuity and quality of service [32]. Regional consistency, on the other hand, concerns the driver: the goal is to repeatedly assign them the same service area, so they can better understand the area and manage visits more efficiently [31]. Finally, customer consistency emphasizes the frequency with which a driver serves the same customer, creating familiarity and optimizing the management of specific customer needs, as occurs in home care [31]. Consistency of delivery, on the other hand, is an important criterion in inventory routing problems (IRPs) and concerns the continuity and predictability of the service provided to the customer. It involves ensuring that quantities delivered are stable over time, that intervals between two consecutive visits are regular, and that inventory levels at the customer remain constant [31].
Alongside similarity and the search for consistent solutions over time, there is also the opposite approach: dissimilarity, that is, the intentional variation of visiting routes or times. This principle is applied when predictability poses a risk, especially in safety contexts such as the transport of dangerous valuables or materials [33]. Making the routes less predictable in fact reduces the opportunities for robbery or attack.
The introduction of consistency, whether temporal, personal, or delivery, into VRP optimization problems has a clear influence on the cost and total distance traveled, generally leading to an increase in such metrics [31]. This increase in cost or distance (routing cost) is considered an acceptable, or even desirable, trade-off, as it is largely offset by benefits such as increased customer satisfaction and improved driver productivity [31].

1.2.3 Multi-objective function

The combination of the two conflicting measures — such as cost or distance (economic dimension) on the one hand and coherence/similarity or dissimilarity (operational, social or security dimension) on the other— is achieved through two main approaches in VRP modeling. The most common approach to handling multiple or conflicting objectives is to integrate the secondary objective in the form of penalties or rewards within the primary cost function [31]. In problems such as the Consistent Vehicle Routing Problem (ConVRP), temporal consistency is crucial. While the primary goal remains minimizing total travel time, the objective function is increased by adding penalties for deviations from the desired consistency [34]. A study on the VRP with Service Choice (PVRP-SC) instead described the objective function as combining travel cost with the benefit derived from quality of service [29]. When objectives, such as cost and consistency or dissimilarity, are treated as independent objectives, bi-objective or multi-objective programming is used. This approach, not adopted in this work, produces a non-dominated set of efficient solutions, known as the Pareto Front [34]. The multi-objective approach better reflects real and complex situations and provides a comprehensive picture of the trade-offs possible in the system under consideration [35].

1.2.4 Optimization algorithms

The Vehicle Routing Problem (VRP) and its generalizations are universally recognized as problems of high combinatorial complexity and are classified as NP-hard. Due to their NP-hard nature, the goal of solving these problems with exact solutions is unrealistic for large arbitrary instances [29]. To solve complex real-world instances, involving elaborate constraints, such as time windows, multiple deposits, and heterogeneous fleets, we mainly resort to heuristics and metaheuristics, which provide high-quality or near-optimal solutions in a reasonable time. Solution methods for VRP are generally divided into: exact methods, approximate algorithms and hybrid algorithms [13]. The former provide optimal solutions but are only suitable for small-scale problems. Approximate Aagorithms produce close-to-optimal (near-optimal) solutions and are essential for large instances. These include heuristics and metaheuristics. Meanwhile, hybrid algorithms combine two or more techniques, such as exact methods, approximate algorithms, simulation, and machine learning (machine learning), and have become the preferred choice for addressing the complexity of WCVRP [13].

Heuristics are approximate algorithms specifically designed to solve certain types of problems. They focus on systematically finding an acceptable solution within a limited number of iterations [13]. These include Construction Algorithms, which generate a path based on construction rules, for example by choosing the node closest to the current node as the next customer to visit,

i.e. the nearest neighbor rule [5]. The heuristic algorithm can therefore be defined as a set of fixed rules that guide the search for a feasible solution that is closest to the optimal possible. Its simplicity allows for quick implementation and application, but tends to get stuck on local optima [5].

Metaheuristics, in their original definition, are solution methods that orchestrate an interaction between local improvement procedures and higher-level strategies [36]. The fundamental goal is to create a process capable of escaping local optima and performing robust search in the solution space. A successful metaheuristic should ensure a balance between two strategies: exploration, which involves investigating the solution space to search for promising areas, and exploitation, which can be defined as intensifying local research to improve an already good solution [36]. Over time, the term "metaheuristics" has come to include any procedure that employs strategies to overcome the trap of local optimality in complex solution spaces [36]. This includes in particular procedures that use one or more neighborhood structures as a means of defining permissible moves to move from one solution to another, or to build or destroy solutions in constructive and destructive processes. The concept of neighborhood is central to metaheuristics, and in particular to Local Search (LS) algorithms, because it determines the structure of the search space and the possible transitions between a solution and its neighboring solutions. In the Handbook of Metsheuris [36], Glover and Kochenberger define the neighborhood as: " the local transformations that can be applied to the current solution, denoted S, define a set of neighboring solutions in the search space, denoted $N(s)$ (the neighborhood of S). Formally, $N(s)$ is a subset of the search space defined by:

$$N(S) = \{\text{solutions obtained by applying a single local transformation to } S\}.$$

The neighborhood is implicitly defined by a set of moves, which are the local transformations that define the transition between configurations. In practice, the neighborhood structure defines the set of solutions that can be reached by the solution S in a single step of a local search algorithm. The choice of search space and neighborhood structure is considered the most critical step in the design of a local search heuristic [36]. The neighborhood function $N(s)$ establishes accessibility relationships between configurations. The number of neighboring solutions that can be generated from a current solution based on a given move is called neighborhood size. It is believed that too small a neighborhood does not allow it to move fast enough in space and may prevent the algorithm from sufficiently exploring the solution space. While one that is too large is reduced to random sampling, reducing its ability to focus on specific areas. The optimal size is very problem-specific [36]. Relocate Neighborhood is an example of Small Neighborhood Search (SNS). The move involves moving the customer to a different location within their current path or another path. Such heuristics are characterized by neighborhood size that grows polynomially and contentfully with respect to instance size [37]. For example, for the CVRP, the relocation neighborhood has a size $O(n^2)$, where n is the number of customers. The destroy and repair method, on the other hand, is the fundamental principle underlying algorithms such as Large

Neighborhood Search (LNS), which explores a neighborhood implicitly and very extensively [37]. In LNS, the neighborhood $N(s)$ is not defined by a simple move, but is implicitly defined by two sequential phases: destroy and repair. The first stage destroys a portion of the current solution, for example, by removing 15% of customers in a CVRP. This operation typically renders the solution unfeasible. Next, the repair method reconstructs the destroyed solution to return it to a feasible shape. To continue the example, removed customers are reinserted using a greedy heuristic. Since the destruction method can destroy a large part of the solution, this neighborhood type typically contains a very large number of solutions. The size and complexity of the neighborhood directly impact execution time. If the neighborhood is too large or complex, the algorithm may take too long to evaluate all neighbors, making the search too expensive [36]. The primary goal of metaheuristics is to provide high-quality solutions in a reasonable time, so it is necessary to find the right balance in the trade-off between computation time and solution quality.

Metaheuristics Based on Single Solution Search (Neighborhood-Based) are extensions of local search algorithms and focus on avoiding getting stuck in local optima, allowing for non-improving moves [37]. Tabu Search (TS) appears in this category, which uses memory structures to avoid cycling between solutions already visited and to balance intensification and diversification. Simulated Annealing (SA), on the other hand, is known for its property of avoiding local optima by probabilistically accepting pejorative moves. The probability of accepting a pejorative move depends on a temperature parameter, which gradually decreases. This parameter ensures exploration in the initial phase of the approach and, as time passes, exploration gives way to greater exploitation.
Variable Neighborhood Search (VNS) exploits the fact that a local optimum relative to one neighborhood is not necessarily a local optimum relative to another [36]. The VNS is based on three empirical and theoretical facts relating to local research [37]. The first is neighborhood dependence: a local minimum relative to one neighborhood structure is not necessarily a local optimum relative to another structure. Contrary to the global optimum, second principle, which remains a local optimum with respect to all possible neighborhood structures. Finally, the proximity of excellent premises is a condition that occurs for many problems. This empirical observation implies that a local optimum often provides useful information about the global optimum. Based on these concepts, we can conclude that using multiple neighborhood structures can help us avoid local optima and explore regions of the solution space that might be closer to the global optimum. However, this does not imply that the point at which the algorithm stops coincides with the global optimum, but only that it represents a local optimum with respect to the neighborhoods considered.

Another class of metaheuristics is population-based (Multi-Start Based) [36]. These methods maintain and evolve a set of solutions or construct solutions probabilistically in multiple iter-

ations. Among these, Genetic Algorithms (GAs) are known as evolutionary algorithms that operate on a population of solutions (chromosomes) and use recombination and mutation operators, inspired by natural selection [36]. Ant Colony Optimization (ACO) is also inspired by nature. The initial method was proposed by Marco Dorigo et al., in 1991 [38], observing the behavior of ants that, following pheromone tracks, incrementally construct solutions to determine the optimal path between their colony and a food source.

Finally, higher-level metaheuristics aim to control or guide other heuristics, thus operating at a higher level of abstraction than traditional optimization techniques. An example is Hyperheuristics (HH), designed to intelligently select or generate the heuristics best suited to the problem. Unlike classical metaheuristics, which directly explore the solution space, hyperheuristics work on a space of heuristics, aiming to obtain more generalizable systems that can be adapted to different types of problems [36]. Another high-level methodology is Reactive Search Optimization (RSO), which integrates machine learning principles into heuristic search. The RSO automatically adjusts the algorithm parameters during execution, through an online learning mechanism that allows the method to dynamically adapt to the problem structure [37].

1.2.5 Literature review on optimization algorithms for PVRP

Metaheuristics have become the preferred solution approach for PVRP and WCVRP to escape local optima and find high-quality solutions for large instances [13]. Tabu Search was applied by Cordeau et al. (1997) [39] to solve multi-depot PVRP, while Alonso F, Alvarez MJ and Beasley JE (2008) [40] used a TS algorithm for PVRP with multiple vehicle trips and accessibility restrictions. Adaptive Large Neighborhood Search (ALNS) is considered one of the most promising metaheuristics. Coelho et al. (2016) [41] used ALNS for the PVRP variant with a service type option (PVRP-SC) for instances of up to 1,500 customers. Aksen, Kaya, Salman, and Tüncel (2014) [42] proposed an ALNS for a selective and periodic inventory routing (IRP) problem, and Chen, Demir, and Huang (2021) [43] developed a hybrid ALNS heuristic for VRP with time windows and delivery robots. The VNS algorithm has been used to address both VRP, such as the study by Kytöjoki et al. (2007) [44], and PVRP, as in the case of Hemmelmayr et al. (2013) [45] who proposed a hybrid VNS-based solution method for PVRP with Intermediate Structures (IF).

Wouda et al. (2013) [46] developed a variant of the Hybrid Genetic Search (HGS) algorithm, a metaheuristic that combines a genetic algorithm with a highly efficient Local Search procedure. The approach is designed to solve both time-windowed VRP (VRPTW) and capacity-constrained VRP (CVRP), and is easily extendable to other variants of the problem. The algorithm employs a population composed of feasible and infeasible solutions to maintain diversity and, at

each iteration, generates new individuals via a crossover operator followed by an intensive local optimization phase.

Chapter 2

Problem description

2.1 Operational context of the problem

This work analyzes the waste collection problem through the Periodic Vehicle Routing Problem (PVRP) framework, with the aim of developing solutions that are efficient and, at the same time, consistent with operational needs and patterns of demand variability. In routine planning, the expected demand for waste generated at each node is estimated. Estimates are generally based on historical data. Vehicle routes and collection days are typically established only once and optimized based on these expected demands. The routes thus determined represent the system's usual collection plan: they are the routes that vehicles follow stably over time and tend to remain unchanged even when changes in operating conditions occur. Therefore, when waste generation patterns fluctuate significantly due to seasons, special events, or other factors, it triggers the condition that, combined with cost minimization or stringent resource constraints, produces inconsistent routing plans. To efficiently cope with fluctuations in collection demand, minimizing costs and distances traveled, the most effective solution would consist of periodic re-optimization of routes. This approach is considered undesirable because collection operators prefer to maintain a routine and consistency of service. Consistency is key to decreasing the learning burden for drivers, and secondly also to improving customer satisfaction. Staff benefit from maintaining a certain routine: the driver becomes familiar with the practical details of the actual road network and the routes to be followed, while the collection operator remembers more precisely the location of the less visible bins or containers. This familiarity allows drivers and teams to work efficiently. Interviews with practitioners confirmed that both driving time and service time increase significantly when drivers operate outside their habitual areas or when nodes are not part of their regular routes. The research goal is to develop new solutions for modified instances, where demand has changed, that deviate as little as possible from the original optimized paths.

This requires exploring different definitions of "similarity" and analyzing the trade-off between similarity and path length.

2.1.1 Problem Workflow

In this study, two types of instances are considered: reference instances (R) and perturbed instances (P), as illustrated in Figure 2.1. Disturbed instances are generated by changing customer demand to reflect fluctuations in waste generation. Both R and P undergo the same initial two-stage optimization process, consisting of an assignment algorithm and a VRP solver, aimed at minimizing the travel distance and producing what we call the primary solution for each case. However, only the perturbed instances move to a third stage: an adaptive multiobjective optimization based on VNS, which refines the primary solution by jointly minimizing the path length and dissimilarity with respect to the reference solution. Figure 2.1 summarizes this workflow, highlighting the parallel structure of the first two phases and the additional refinement phase applied exclusively to the perturbed instances. The algorithms involved in these steps are described in detail in Chapter 4.

2.2 Reference and perturbed instances

This section describes the two types of cases considered in this study. Section 2.2.1 covers reference instances used as a baseline, while Section 2.2.2 examines demand fluctuation modeling for perturbed instances.

2.2.1 Reference instances

The VRP literature makes extensive use of shared instances to allow direct comparison between different solution approaches. For this reason, standard instances for PVRP are also adopted in this study. In particular, the so-called 'old data set' containing instances p01–p32 is used, which constitutes the reference dataset for generating routing solutions. Specifically, instances p01–p10 were originally proposed by Eilon et al. [47] and adapted to PVRP by Christofides and Beasley [48]; p11 by Russel and Igo [49]; p12–p13 by Russel and Gribbin [50]; and p14–p32 by Chao et al. [51]. The instances, originally proposed by various authors, were collected and made accessible online by Cordeau et al. [52].
Table 2.1 presents the main parameters of the instances. Each instance specifies the number of

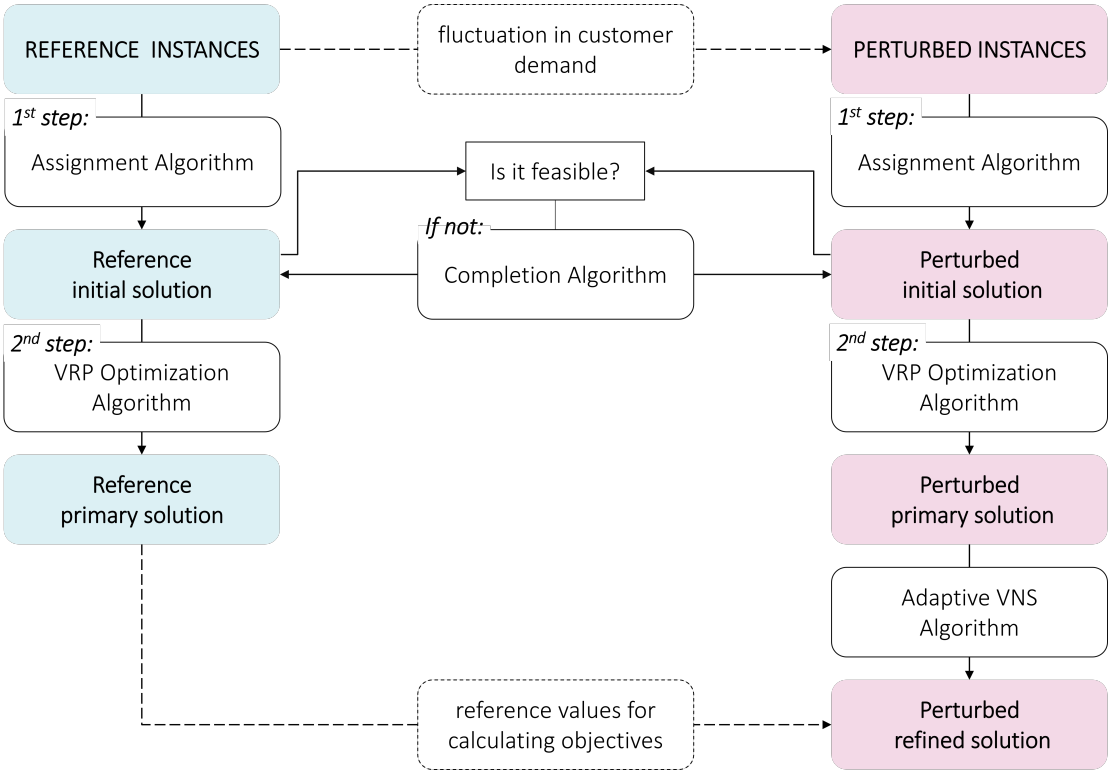


Figure 2.1: Workflow of the solution generation process.

clients served N, the number of days T in the planning period, and the vehicles available K with their respective capabilities Q. Each customer is associated with a geographical location, demand and frequency of service to be respected. Frequency determines the set of possible combinations of days on which the customer can be visited. In the Table, the frequency service columns f_i indicate how many customers request a minimum frequency equal to i. There are no service time constraints and therefore the maximum duration of a path D is always zero.

2.2.2 Demand fluctuation for perturbed instances

Disturbed instances were obtained by modifying customer demand. A previous study addressed a similar problem, analyzing how an assignment rule can increase familiarity between drivers and customers, with the aim of improving efficiency and quality of service [53]. To evaluate its behavior in different contexts, the authors generated various scenarios by modifying various instance factors, including customer demand. In simulation experiments, customer demand was modeled using the Gamma distribution. This choice was motivated by the need to explore

Instance	N	K	T	D	Q	service frequencies					
						f1	f2	f3	f4	f5	f6
p01	50	3	2		160	50					
p02	50	3	5		160	17	26			7	
p03	50	1	5		160	50					
p04	75	2	5		140	75					
p05	75	6	5		140	30	34			11	
p06	75	1	10		140	75					
p07	100	4	2		200	100					
p08	100	5	5		200	40	46			14	
p09	100	1	8		200	100					
p10	100	4	5		200	40	46	14			
p11	139	4	5		235	103	22	12	1	1	
p12	163	3	5		140	148	8	7			
p13	417	9	7		2000	377	40				
p14	20	2	4		20	8	8		4		
p15	38	2	4		30	16	16		6		
p16	56	2	4		40	24	24		8		
p17	40	4	4		20	16	16		8		
p18	76	4	4		30	32	32		12		
p19	112	4	4		40	48	48		16		
p20	184	4	4		60	80	80		24		
p21	60	6	4		20	24	24		12		
p22	114	6	4		30	48	48		18		
p23	168	6	4		40	72	72		24		
p24	51	3	6		20	36	9				6
p25	51	3	6		20	36	9				6
p26	51	3	6		20	36	9				6
p27	102	6	6		20	72	18				12
p28	102	6	6		20	72	18				12
p29	102	6	6		20	72	18				12
p30	153	9	6		20	108	27				18
p31	153	9	6		20	108	27				18
p32	153	9	6		20	108	27				18

Table 2.1: Instance characteristics and service frequencies.

a wide range of variations around a fixed mean, while ensuring that all generated values were positive, thus avoiding the emergence of negative demands [53]. Following their work, the Gamma distribution is adopted in this study to model fluctuations in customer demand.

Analyzing the factors that influence waste demand, different behaviors can be observed. If waste collection services are suspended, such as during snowfall or holidays, customer demand is generally expected to increase. The trend in individual applications could be relatively uniform with varying amounts depending on the type of clientele. Differently, seasonal changes can lead to both an increase or a decrease in overall demand, while individual customers may exhibit different behaviors. As discussed, during the summer, overall waste generation tends to increase, but residents may produce less due to vacations, while restaurants and tourism businesses may experience greater demand due to increased tourism activity. In this study, to ensure the existence

of viable solutions, overall demand was kept constant. This allows individual customer demand to vary up or down, provided the total remains in equilibrium.

2.3 Conceptual consistency metrics

The aim of this study is to evaluate Person-Oriented Consistency, more precisely the operational consistency observed from the perspective of collection service operators. The similarity to be obtained between the reference paths and those of the perturbed instances can be interpreted in two different ways, depending on the type of operator involved. In fact, a collection system is envisaged in which at least two figures operate on each vehicle: the employee responsible for collecting the bins from the buildings or collection points and taking them to the truck for emptying, and the driver who drives the vehicle along the route.
The operator responsible for picking containers prioritizes continuity in serving the same set of customers. Consequently, for this professional figure the similarity between the reference problem and the perturbed instances is evaluated over the entire period, regardless of the correspondence between the days of service. In other words, the operator develops a memory referring to the overall set of customers served, without attributing relevance to the fact that a given customer is visited on the same day of the week in the different configurations of the problem. Differently, the driver can develop a specific memory of individual daily routes (for example: the typical Monday route, the Tuesday route, and so on). In this case, consistency is analyzed by comparing the paths of the reference problem with those of the perturbed instance day by day. The similarity between the paths is then evaluated on the basis of the edges actually traveled, according to two criteria: the maximum length of the corresponding subsequence and the total number of common edges.

2.3.1 Definition of the aggregated objective function

In this research, the two measures of interest — the distance traveled and the similarity between the customer sets and the paths of the reference problem and those of the perturbed instances — were combined within a single objective function. This choice is motivated by the need to obtain a synthetic and directly comparable solution between different scenarios, avoiding generating an entire Pareto front and making the analysis of the impact of perturbations on the operational quality of the system more immediate. Integrating the two dimensions into a single index also allows for a compact assessment of the trade-off between economic efficiency and operational consistency, maintaining the problem structure in the form of minimization.
Both measures are expressed in normalized form with respect to the reference solution, but with different scales. The relative distance is defined as the ratio between the distance of the per-

turbed instance and that of the base solution and, in this way, a relative distance equal to 1 indicates that the perturbed instance has a travel cost identical to that of the base solution. The similarity, however, varies in the interval $[0, 1]$, where higher values indicate a greater correspondence between the perturbed solution and the reference one according to the adopted metric. It should be emphasized that, since different similarity measures are adopted, a value equal to 1 does not necessarily imply that the two solutions are identical, as each metric captures different aspects of the route structure. The precise formalization of these measures will be presented in Chapter 3.

Chapter 3

Mathematical formulation

3.1 Period Vehicle Routing Problem formulation

The mathematical model adopted in this work to describe the Periodic Vehicle Routing Problem (PVRP) derives from the formulation proposed by Coene et al. (2010) [28], which constitutes a consolidated reference in the literature. The general structure of the model was maintained, including the definition of the sets, parameters, and variables needed to represent the choice of visit scenarios, the construction of daily routes, and compliance with vehicle capacity constraints. However, compared to the original formulation, some modifications have been introduced in order to adapt the model to the characteristics of the instances used in this research. First, since those instances do not provide for disposal nodes, the entire component relating to disposal facilities has been removed. Furthermore, since there are no travel time limits or time windows, all parameters, variables, and constraints related to the temporal aspects of Coene et al.'s model have been eliminated. Finally, each vehicle is assumed to start its activity each day with zero load, a condition consistent with the structure of the considered instances and the absence of restrictions on the residual quantity transported at the beginning of the day. The resulting formulation therefore represents a simplified version but fully compliant with the needs of the problem analyzed, while maintaining the conceptual framework of periodic modeling proposed by the original authors.

We define the transport network as an indirect and complete graph $G = (V, A)$. The set of vertices is $V = \{0, 1, \ldots, N\}$, where node 0 represents the depot and nodes $1, \ldots, N$ constitute the set of customers, which we denote by $\mathcal{N} = \{1, \ldots, N\}$. The set of edges is A, and for each edge $(i, j) \in A$ a symmetric distance is defined ($d_{ij} = d_{ji}$). The planning period extends over

T days, collected as a whole $\mathcal{T} = \{1, \ldots, T\}$. Each customer must be visited according to a scenario $c \in \mathcal{C}$, where $\mathcal{C}$ is the set of eligible scenarios. A scenario is defined as a subset of days within T; it specifies on which days a customer is to be served. Let f_c indicate the number of visit days foreseen by scenario c. There are K vehicles available, each equipped with a capacity Q_k. For each customer $i \in \mathcal{N}$, the required frequency f_i and the quantity to be collected at each service q_i are also given. We also define the binary variable a_{ct}, equal to 1 if the day $t \in \mathcal{T}$ is included in the scenario $c \in \mathcal{C}$, and 0 otherwise.
The decision variables adopted in the model are as follows. Let x_{ijkt} be a binary variable that takes the value of 1 if the vehicle $k \in \mathcal{K}$ visits node $i \in V$ and immediately after node $j \in V$ on day $t \in \mathcal{T}$, and the value 0 otherwise. Furthermore, let y_{ic} be a binary variable that holds 1 if the customer $i \in \mathcal{N}$ is assigned to scenario $c \in \mathcal{C}$, and 0 otherwise. To represent vehicle load, we also introduce the variable L_{ikt}, which indicates the total vehicle load $k \in \mathcal{K}$ on day $t \in \mathcal{T}$ after visiting customer $i \in V$. This variable allows you to monitor vehicle capacity and ensure that the maximum amount that can be transported during service is not exceeded.

The optimization function

$$\text{Minimize} \sum_{i \in V} \sum_{j \in V} \sum_{k \in \mathcal{K}} \sum_{t \in \mathcal{T}} d_{ij} x_{ijkt} \tag{3.1}$$

subject to

$$\sum_{c \in \mathcal{C}: f^c = f_i} y_{ic} = 1, \quad \forall i \in \mathcal{N} \tag{3.2}$$

$$\sum_{j \in V} \sum_{k \in \mathcal{K}} x_{ijkt} - \sum_{c \in \mathcal{C}} a_{ct} y_{ic} = 0, \quad \forall i \in \mathcal{N},\ \forall t \in \mathcal{T} \tag{3.3}$$

$$\sum_{i \in V} x_{ihkt} - \sum_{j \in V} x_{hjkt} = 0, \quad \forall h \in V,\ \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.4}$$

$$\sum_{j \in \mathcal{N}} x_{0jkt} \leq 1, \quad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.5}$$

$$\sum_{i \in S} \sum_{j \in S} x_{ijkt} \leq |S| - 1 \qquad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T},\ S \subseteq \mathcal{N},\ |S| \geq 2 \tag{3.6}$$

$$(L_{ikt} + q_j - L_{jkt}) \leq (1 - x_{ijkt}) Q_k \qquad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T},\ \forall i, j \in \mathcal{N} \tag{3.7}$$

$$L_{ikt} \leq Q_k \qquad \forall i \in \mathcal{N},\ \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.8}$$

$$L_{0kt} = 0 \qquad \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.9}$$

$$x_{ijkt} \in \{0,1\} \qquad \forall i,j \in V,\ \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.10}$$

$$y_{ic} \in \{0,1\} \qquad \forall i \in \mathcal{N},\ \forall c \in \mathcal{C} \tag{3.11}$$

$$L_{ikt} \geq 0 \qquad \forall i \in V,\ \forall k \in \mathcal{K},\ \forall t \in \mathcal{T} \tag{3.12}$$

The model assigns a visit scenario to each customer and determines a set of daily routes over the planning horizon, in order to minimize the total distance traveled. Equation (3.1) defines the objective function of the basic PVRP model, where the total distance traveled in the planning period is optimized. In particular, the goal is to minimize the sum of the lengths of all the arcs crossed by each vehicle on each day of the period. Constraints (3.2) ensures that exactly one service scenario is selected for each customer and that this choice is consistent with the assigned visit frequency. Once the scenario has been determined, the constraints (3.3) ensures that the customer is actually served on the days foreseen by the selected scenario. The consistency of routes is ensured by constraints (3.4), which requires that each arrival at a customer by a vehicle is followed by the relevant departure. Constraints (3.5) limits the use of each vehicle to a single route per day, preventing the same vehicle from performing multiple daily routes. Equation (3.6) represents the subtour elimination constraints, which prevents the formation of cycles not connected to the depot and ensures route connectivity. Monitoring of transported loads is managed by constraints (3.7), which regulates the updating of the vehicle load after each visit, while constraints (3.8) requires that the load does not exceed the maximum available capacity. In the original model, constraints (3.9) requires that a vehicle be unloaded when visiting a disposal facility. In this work, since the instances do not provide for such systems and the depot implicitly performs the function of an unloading point, the constraints is modified to require that each vehicle start the day with zero load. Finally, equations (3.10), (3.11) and (3.12) impose binary conditions and non-negativity conditions for the decision variables, respectively, ensuring the correct discrete nature of the problem.

3.2 Demand fluctuation modeling

The change in demand for each customer q_i is generated using a Gamma distribution, which is defined by the function:

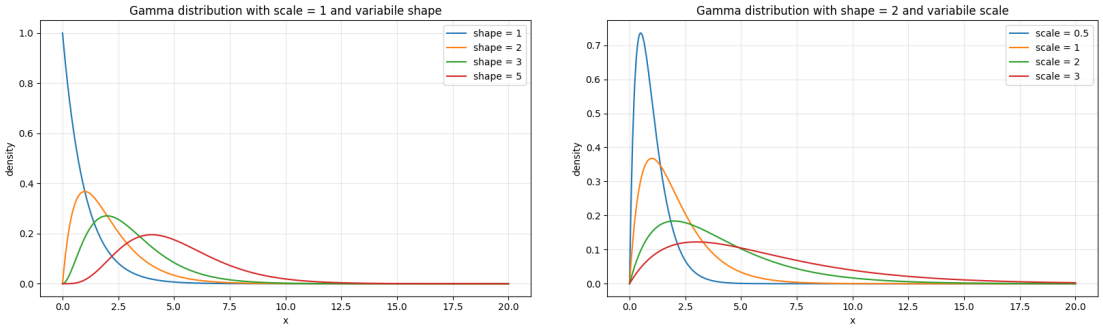
$$\Gamma(\alpha) = \int_0^\infty y^{\alpha-1} e^{-y}\, dy, \qquad \text{per } \alpha > 0. \tag{3.13}$$

The Gamma distribution is a probability distribution defined over the positive real numbers, $\mathbb{R}^+$, and is parametrized by the pair of positive numbers (α, β), where α is the shape parameter and β is the scale parameter.

As shown in the Figure 3.1a, varying the shape parameter α while keeping β fixed significantly alters the structure of the Gamma distribution, affecting its symmetry, concentration of mass, and relative dispersion. To ensure consistency across customers and scenarios, it is therefore more appropriate to keep α fixed and vary only β. In this way, as can be seen from the Figure 3.1b, the distribution preserves the same overall shape for all customers, while the spread of the curve scales proportionally to their initial demand, since the expected value X_i satisfies:

$$\mathbb{E}[X_i] = \alpha \cdot \beta_i \tag{3.14}$$

This results in realistic behavior: customers with higher demand naturally exhibit greater absolute variability, while customers with lower demand show smaller fluctuations. This approach introduces uncertainty in a proportional and statistically consistent manner.



(a) Gamma distribution with fixed β

(b) Gamma distribution with fixed α

Figure 3.1

Choosing a shape parameter $\alpha = 2$ reflects a level of variability consistent with real-world phenomena characterized by moderate skewness and strictly non-negative values, such as daily customer demands.

The new demand for customer i, x_i, is defined by the probability density function:

$$f(x_i \mid \alpha, \beta_i) = \frac{1}{\Gamma(\alpha)\,\beta_i^{\alpha}}\, x^{\alpha-1} e^{-x_i/\beta_i}, \qquad x_i > 0. \tag{3.15}$$

To avoid unrealistically low values, a minimum threshold of (10%) of the original demand is imposed:

$$X_i^{new} \geq 0.1\, q_i \tag{3.16}$$

The total demand of all customers is kept unchanged, meaning the demand varies in a compensated manner:

$$\sum_i (X_i^{\text{new}} - q_i) = 0 \tag{3.17}$$

Modified demands are rescaled to preserve the total sum:

$$\sum_i X_i^{\text{new}} = \sum_i q_i \tag{3.18}$$

This choice ensures the feasibility of the routing problem, preventing the total assigned demand from exceeding the available vehicle capacity.

3.3 Formalization of consistency metrics and aggregated objective

Consistency is considered exclusively for perturbed instances, since the goal of the model is not limited to minimizing the total distance traveled, but also aims to preserve continuity with respect to the reference solution.

To formally introduce consistency measures, it is necessary to define some new notations relating to the routes present in the reference solution s and in the perturbed solution s'. The apostrophe $(\cdot')$ always indicates entities belonging to the perturbed solution. For each day $t \in \mathcal{T}$ and for each vehicle $k \in \mathcal{K}$, we indicate with r_{kt} the route taken by vehicle k on day t. To each vehicle k we associate the set of customers actually served, denoted by U_k. To each route r_{kt}, we use the notation E_{kt} to define the set of all the edges which constitute the route. To describe the sequential aspect of visits, we define S_{kt} as the set of all feasible subsequences of the route r_{kt}. Similarly we will have $U'_{k'}$, $E'_{k't}$ and $S'_{k't}$ for the routes $r'_{k't}$ belonging to the perturbed solution. The dimensions of these sets play a fundamental role in the normalizations that will intervene

in the formulations of consistency measures: $|U_k|$ represents the number of customers visited in the reference route and $|E_{kt}|$ corresponds to the number of arcs that compose it.
It should be noted that the path r_{kt} of the reference solution, relative to the vehicle k on day t, does not necessarily correspond to the path $r'_{k't}$ of the perturbed solution assigned to the same index $k = k'$ and the same day t. To establish correspondences between vehicles in the two solutions, all possible pairings between vehicles of the reference solution $(k_1, k_2, \dots)$ and those of the perturbed solution $(k'_1, k'_2, \dots)$ are considered. The optimal pairings are those that maximize total similarity. Once such couplings have been determined, the correspondence between routes can then be defined. For example, if the optimal coupling involves the correspondence between vehicle k_1 and vehicle k'_3, then on day $t = 0$ the route $r_{k=1,t=0}$ corresponds to $r'_{k'=3,t=0}$, on day $t = 1$ we have $r_{k=1,t=1}$ corresponding with $r'_{k'=3,t=0}$, and so on for all the days of the period.

Within this framework, three different metrics are defined, each of which captures a specific aspect of the correspondence between the perturbed and the reference solution: customer-based consistency, sequential route structure, and arcs traveled. The mathematical formalization of these measures is presented in paragraphs 3.3.1, 3.3.2 and 3.3.3. Subsequently, in paragraph 3.3.4, the aggregate objective function is illustrated, which consistently integrates the economic dimension, represented by the distance traveled, and the operational dimension, represented by consistency, thus allowing us to evaluate the trade-off between routing efficiency and customer and route similarity.

3.3.1 Nodes-set consistency

Definition 3.3.1 (Nodes-set consistency)**.** We define the set of routes served by a vehicle as similar to the set of routes served by another vehicle if it contains as many identical nodes as possible, regardless of the sequence.

The *nodes consistency* between vehicles k and k' is evaluated by comparing the set of customers served U_k by the vehicle of the reference solution with the set of customers served by the vehicle of the perturbed solution $(U'_{k'})$. This measure is defined as the ratio between the number of customers shared by the two vehicles $(|U_k \cap U'_{k'}|)$ and the total number of customers $|U_k|$ of the vehicle k.

$$sim_{k,k'} = \frac{|U_k \cap U'_{k'}|}{|U_k|} \tag{3.19}$$

Vehicle similarity can take values in the range $[0, 1]$, where the value 1 indicates that the entire set of customers served by vehicle k in the reference solution is served, in the perturbed solution, by vehicle k'. Note that $U'_{k'}$ may include additional customers not belonging to the k routes.

To compare the solutions s and s' it is necessary to define the *nodes-set consistency* between solutions. The minimum of all similarity measures between pairs of vehicles is assumed as the solution similarity value. This choice allows you to optimize the worst value, that is, to increase the minimum threshold.

$$sim_{s,s'} = \min_{k \in \mathcal{K},\, k' \in \mathcal{K}'} (sim_{k,k'}) \tag{3.20}$$

As the document continues, this measure will be indicated with the acronym sim_1.

3.3.2 Arc-sequences consistency

Definition 3.3.2 (Arc-sequences consistency)**.** We define a route as similar to another route if it contains an identical, uninterrupted sequence, the longer the better.

The measure of *arc-sequences* requires a new notion: it is in fact necessary to define the longest uninterrupted sequence generated by the intersection of the sub-routes served in the route r_{kt} and those visited in the route $r'_{k't}$.

$$\text{seq}_{\max}^{k,k',t} := \arg \max_{\sigma \in \left(S_{kt} \cap S'_{k't}\right)} |\sigma| \tag{3.21}$$

The length of this sequence is:

$$\ell_{\max}^{k,k',t} := \max_{\sigma \in (S_{kt} \cap S'_{k't})} |\sigma| \tag{3.22}$$

Given the maximum common sequence, the similarity between the vehicle routes k and k' on day t is defined as the ratio of the length of the sequence to the number of arcs traveled in the reference route.

$$sim_{k,k',t} = \frac{\ell_{\max}^{k,k',t}}{|E_{kt}|} \tag{3.23}$$

The value of the measure varies in the range $[0, 1]$, where a similarity of 1 indicates that the path originally followed by vehicle k on day t coincides exactly with the deposit-to-deposit path performed by vehicle k' in the perturbed solution. The measure is calculated in both drivable directions of the route, since the direction is not imposed, and only the direction with the highest similarity value is taken into account.

The similarity between vehicles k and k' is defined as the average of the similarity values between their routes. The use of the average allows to obtain a value representative of the overall similarity

of the vehicle.

$$sim_{k,k'} = \underset{k \in \mathcal{K},\, k' \in \mathcal{K}',\, t}{\text{mean}} \left(sim_{k,k',t} \right) \tag{3.24}$$

Finally, the *arc-sequences consistency* between the solutions s and s' is defined, as in the previous measures, as the minimum value of the similarity between vehicles, as expressed in (3.20). This measurement will from now on be identified with the acronym sim_2.

3.3.3 Arcs-set consistency

Definition 3.3.3 (Arcs-set consistency)**.** We define a route as similar to another route if it contains as many identical arcs as possible, irrelevant of the sequence.

The *arcs-set consistency* between the routes r_{kt} and $r'_{k't}$ is evaluated by comparing the set of arcs traveled by the vehicle in the reference solution (E_{kt}) with the set of arcs traveled ($E'_{k't}$) by the vehicle in the solution perturbed on the same day. This measure is defined as the ratio of the number of shared arcs ($|E_{kt} \cap E'_{k't}|$) to the total number of arcs $|E_{kt}|$ of the vehicle k.

$$sim_{k,k',t} = \frac{|E_{kt} \cap E'_{k't}|}{|E_{kt}|} \tag{3.25}$$

The value of the measure varies in the range $[0, 1]$, where 1 indicates that all arcs originally traveled by vehicle k on day t also appear in the path of vehicle k' in the perturbed solution. It is still possible that the route $r'_{k't}$ includes arcs further than r_{kt}. Again, the measure is calculated for both directions of the route, since the direction is not constrained.

Similar to *arc-sequences consistency*, the similarity between vehicles k and k' is equal to the average value of the similarities calculated for all routes belonging to the same pair of vehicles over the entire period. See the equations (3.24).

The last measure considered is the *arcs-set consistency* between solutions, which is also defined, as in the previous measures, by the equations (3.20). This last measurement will instead be called via the abbreviation sim_3.

3.3.4 Weighted objective function

The measures of *consistency* and *distance* are combined in order to construct the objective function using appropriate weights. In the equations (3.1) the objective function of the reference problem had been defined, in which only the distance was minimized. This value is now used to normalize the total distance traveled in the perturbed solution.

We therefore define the total distance of the solution s as the sum of all the arcs belonging to the routes traveled by the vehicles in the entire period:

$$D_s := \sum_{i \in V} \sum_{j \in V} \sum_{k \in \mathcal{K}} \sum_{t \in \mathcal{T}} d_{ij} x_{ijkt} \tag{3.26}$$

Similarly, we get $D'_{s'}$ for the perturbed solution. The normalized distance value is then defined as:

$$\delta_{s,s'} := \frac{D'_{s'}}{D_s} \tag{3.27}$$

This measure can take values in the interval $(0, \infty)$. A value of 1 indicates that the two solutions are equivalent in terms of distance traveled. Values above 1 denote an increase in distance in the perturbed solution compared to the reference one, while values below 1 indicate an improvement.

The aim of the study is to simultaneously evaluate distance minimization and similarity maximization. To this end, the value of *dissimilarity* is defined to combine the two measures into a single minimization function. Regardless of the measure of *consistency* considered and given that the maximum similarity value is equal to 1, the *dissimilarity* is defined as:

$$\phi_{s,s'} := 1 - sim_{s,s'} \tag{3.28}$$

Thanks to these definitions, it is now possible to formulate the objective function as the weighted sum of the normalized distance and the *dissimilarity*:

$$\text{Minimize } w_\delta \cdot \delta_{s,s'} + w_\phi \cdot \phi_{s,s'} \tag{3.29}$$

with constraints

$$w_\delta + w_\phi = 1.$$

Chapter 4

Solving approaches

In this section, all algorithms used for generating solutions are presented, following the progression from *initial* solutions, generated by the *Assignment Algorithm*, through *primary* solutions obtained via *VRP Optimization*, to *refined* solutions produced by the *A-VND Algorithm*, which further refine the primary perturbed solutions based on their similarity to the reference solutions. Appendix A presents the Python code for the two main algorithms employed in this study, namely the *Assignment Algorithm* and the *A-VND Algorithm*. These implementations form the operational foundation of the study.

4.1 Primary solution

The approach used to define the *primary solution* is applied to both reference and perturbed instances, as illustrated in Figure 2.1. Given the dual nature of the problem, divided into *routing* and assignment components, the method consists of two distinct phases: an *Assignment Algorithm* and a *VRP Optimization Algorithm* performed separately for each day of the period. These components are described in paragraphs 4.1.1 and 4.1.3. In case the *Assignment Algorithm* fails to assign all customers to their respective routes, and the solution is therefore not *feasible*, a remedial algorithm has been developed, the *Completion Algorithm* is illustrated in Section 4.1.2.

4.1.1 Assignment Algorithm

The *Assignment Algorithm* aims to assign all customers to routes, each defined by a vehicle and a specific day. This is a heuristic algorithm developed for the Periodic Vehicle Routing Problem.

Compared to the mathematical formulation described in Section 3.1, an additional constraint has been introduced that imposes an exclusivity relationship between customer and vehicle: each customer can in fact be assigned to only one vehicle for all days belonging to the selected visit combination. This constraint was introduced to reduce the complexity of the problem by limiting the number of possible combinations and thus simplifying the assignment method. However, this choice can also lead to a criticality, since a customer may not be assignable to any combination of visits if the vehicles' capacity is not sufficient to meet the related demand.

In the process of assigning customers to routes, two aspects are fundamental: on the one hand, the maximum capacity available for each vehicle and, on the other, the frequency of visits requested by customers together with the relevant eligible visit combinations. To effectively manage these constraints, an assignment order based on a hierarchy of priorities has been defined. The idea behind this choice is to first assign customers with more stringent constraints, so as to prevent them from being left without assignment in the final stages of the algorithm, when the remaining capacity of the vehicles is limited. In this context, those with high frequency and significant demand are considered more complex customers, while those with a reduced number of visits or relatively low demand are more flexible. This strategy maximizes the likelihood of obtaining a feasible solution, reducing the risk of conflicts between vehicle capacity and visitation obligations.

For each customer, the algorithm evaluates all possible assignments to vehicles and days, checking whether the remaining capacity of the vehicles allows to meet the demand and whether the combinations of visits respect the required frequency. The routes are constructed using a proximity matrix, which sorts customers based on their distance from each other, allowing efficient initial routes to be generated before optimization. Subsequently, the distances associated with each combination of visits are calculated and the solution that minimizes the total distance traveled by the fleet is identified. The output matrices are updated with optimal assignments, vehicle loads, and route distances, including the deposit at the beginning and end of each route. In this way, the algorithm ensures a balance between capacity constraints, minimum visit frequency, and total distance minimization, while reducing the risk of complex customers remaining unassigned in the final stages of the procedure. The fundamental operating sequence of the *Assignment Algorithm* is presented in Algorithm 1.

Algorithm 1: Compact customer assignment and route optimization algorithm

Input: Total number of customers, vehicle capacity, other parameters

Output: Complete solution with paths, loads, unassigned customers, and total distance

```
Initialize output matrices and algorithm parameters;
for each customer i = 1 to total_number_customers do
    Evaluate the hypothetical assignment of customer i to all vehicles and days;
    Update vehicle loads and capacity constraints;
    Define eligible visit combinations and check the minimum frequency;
    if no valid combination then
        Add i to the unassigned list;
        continue;
    end
    Filter invalid combinations;
    Calculate the distances of routes and combinations;
    Select the optimal combination and update the output matrices;
end
Add the deposit to the beginning and end of each path;
Calculate the total distance traveled;
return complete solution;
```

4.1.2 Completion Algorithm

The *Assignment Algorithm* allows us to determine an initial solution to the problem, which however may not be *feasible*. Among the outputs is the list of unassigned customers. The main cause of such non-assignments lies in the imposition of the exclusivity constraint between customer and vehicle. To overcome this limitation, the *Completion Algorithm* was developed, whose goal is to assign the remaining customers by eliminating the exclusive assignment constraint to the vehicle.

For each customer on the unassigned list, all associated visit combinations are considered; for each of them, the possible vehicle–day assignments are evaluated. If at least one combination proves capable of meeting the customer's frequency and demand requirements, the latter is assigned directly. Otherwise, the visit combination that guarantees the greatest number of available days is selected. For each unavailable day, an already assigned and more flexible customer than the current customer is temporarily removed, in order to free up the necessary capacity. The removed customer is subsequently reassigned following the same procedure. The key steps of the *Completion Algorithm* are detailed in Algorithm 2.

Algorithm 2: Completion algorithm for assigning previously unassigned customers

Input: generic data, customer data, distance and closeness matrices, initial solution
Output: Updated solution with assigned customers

```
Initialize the list of pending customers;
while there are pending customers do
    Extract the next customer and generate all feasible visit schedules;
    Attempt a direct assignment by checking daily vehicle capacities;
    if a full schedule is feasible then
        Assign the customer accordingly;
        continue;
    end
    Select the schedule with the highest number of feasible days;
    For each infeasible day, identify and remove a suitable customer to free capacity;
    Add the removed client to the list of pending customers;
    Assign the customer using the updated availability;
end
return updated solution;
```

4.1.3 VRP Optimization Algorithm

The *VRP Optimization Algorithm* aims to optimize vehicle routes daily, repeating this procedure for each day of the time horizon considered. The approach adopted is based on the Hybrid Genetic Search (HGS) algorithm developed by Wouda et al. (2013) [46] and made available via the Python package PyVRP, an extensible, well-documented and easy-to-use VRP solver.

HGS combines a genetic algorithm with high-performance local search. The genetic component manages a population of both feasible and non-feasible solutions, generating new solutions at each iteration via crossover and introducing sufficient diversity to avoid premature convergence. Each generated solution is then improved through very efficient local search, where capacity and time window constraints are treated as soft constraints and adaptively penalized. The use of a granular neighborhood allows to reduce the complexity of the research while maintaining a good quality of the solutions explored. At the end of each iteration, the improved solution is reintroduced into the population, while any redundant solutions are eliminated to preserve quality and diversity. The process continues until a stopping criterion based on time, number of iterations, or no improvement is met.

Thanks to the integration between genetic phase and local exploration, HGS represents an effective method for solving complex routing problems and provides a solid support for daily route

optimization in the developed model.

4.2 Refined solution

The solution obtained from the perturbed instances and subsequently refined on the basis of distance and similarity with respect to the primary reference solution is defined as *refined solution*. The optimization algorithm adopted to determine it was initially the *Variable Neighborhood Descent* (VND), a variant of the better known Variable Neighborhood Search (VNS) characterized by a hierarchical structure among the different neighborhoods. This algorithm is described in Section 4.2.1. The selected *neighborhoods*, chosen based on the dual nature of the problem and the structure of the PVRP, are illustrated in Section 4.2.2. However, since the hierarchy was not very marked due to the substantial equivalence in terms of efficiency between the different neighborhoods, an adaptive mechanism for their selection was introduced. This modified version of the VNS, which integrates a dynamic choice of neighborhoods, is called by us *Adaptive Variable Neighborhood Search* (A-VNS) and is presented in Section 4.2.3.

4.2.1 Variable Neighborhood Descent Algorithm

Variable Neighborhood Search aims to obtain optimal or near-optimal solutions efficiently and robustly [36]. The basic idea is the systematic change of neighborhood within local research. *Variable Neighborhood Descent* is a fundamental component of the VNS family of metaheuristics and represents a deterministic improvement step based on the idea that local optimality depends on the neighborhood used [37]. Unlike traditional Local Search procedures, which employ one or two neighborhoods, the VND systematically explores an ordered set of neighborhood structures. The algorithm starts from the narrowest neighborhood and searches for the best adjacent solution. Each time an improvement is identified, the solution is updated and exploration restarts from the first neighborhood, so as to progressively refine the local optimum. If, however, one neighborhood does not produce improvements, the algorithm moves on to the next. The process ends when none of the neighborhoods considered allows further advances, thus leading to a local optimum with respect to the entire set of neighborhoods.

Formally, the VND considers an ordered family of neighborhoods $\{N_1(x), N_2(x), \ldots, N_{k_{\max}}(x)\}$, and iteratively searches, for each k, for the best neighbor of the current solution x:

$$x' = \arg \min_{y \in N_k(x)} f(y).$$

The update step occurs according to the rule:

$$(x, k) \leftarrow \begin{cases} (x', 1), & \text{if } f(x') < f(x), \\ (x,\ k+1), & \text{otherwise,} \end{cases}$$

which represents the classic mechanism of *neighborhood change*: in the presence of an improvement, the search returns to the first neighborhood; in the absence, it moves on to the next. The procedure ends when

$$k > k_{\max},$$

that is, when the solution is a local optimum with respect to all the neighborhoods considered. The corresponding code was taken from the Handbook of Metaheuristics (2010) [37] and is described in the Algorithm 3.

Algorithm 3: Variable neighborhood descent [37]

Input: $x, k_{\max}$
Output: x

```
k ← 1;
repeat
    x' ← arg min_{y∈N_k(x)} f(y) // Find the best neighbor in N_k(x);
    x, k ← NeighborhoodChange(x, x', k) // Change neighborhood;
until k = k_max;
return x;
```

A crucial aspect of the VND concerns the order of the neighborhoods, which directly affects the behavior of the algorithm. Typically, they are arranged in order of increasing "force", so that the simplest and least disruptive moves are attempted first, while the more drastic ones intervene only when the previous ones do not bring improvements. This principle allows us to introduce only the minimum level of perturbation necessary to exit a local optimum, promoting a balance between intensification and diversification. Although the most common ordering is based on the size or nature of the movement, there is no universally optimal order: in some contexts, in fact, more complex neighborhoods may be more effective if applied earlier than others. Furthermore, dynamic variants have been studied in which the order is adapted based on the observed performance, while the VND, in its canonical form, remains a deterministic procedure.

4.2.2 Neighborhoods

Neighborhood is an essential concept in Local Search (LS) and metaheuristics in general, as it provides the topological structure of the solution space. The neighborhood of a solution x (denoted $N(x)$) is a subset of the search space that includes all solutions that can be reached by the current solution x by applying a single local transformation. It is often defined implicitly through a set of moves, which determine the possible transitions in the search space.

In this study, several neighborhood structures were adopted, including operators that change the ordering of customers within a single route, moves that involve the simultaneous transfer of one or more customers from one route to another, and operators based solely on similarity. The nomenclature used for neighborhoods based on *move* or *swap* operations follows the form *u_operation_v*. In this structure, the parameter u indicates the element that remains unchanged during the operation, such as the day, vehicle, or visit frequency in the case of customer exchanges, while v identifies the parameter that actually varies, such as the day of service or the assigned vehicle. The *operation* component specifies the type of intervention performed: in *moves* it concerns a single customer, while in *swaps* it involves a pair of customers; more complex variants are also foreseen, such as *multi-swaps*, which operate on groups of customers extracted from one or more routes, and *route-swaps*, which affect the entire route rather than individual positions. In these neighborhoods, a customer is placed on a new route through a mechanism based on proximity to customers already present on the route. To this end, a matrix of neighborhoods between nodes is preliminarily defined. Whenever the set of customers assigned to a route is changed, the route is then re-optimized by updating the order of visits based on mutual distances, so as to maintain a consistent and efficient sequence after each *move* or *swap* operation. Finally, the neighborhoods that act exclusively on the internal structure of a single route correspond to the *2-opt* and *3-opt* operators, used to improve the customer sequence without altering the assignment to different days or vehicles. Each of these structures will be described in the following sections.

It should be emphasized that neighborhoods that allow customers to move from one day to another can generate solutions that violate the visit combinations associated with individual customers, as required by the PVRP. These situations are managed using a visit reallocation algorithm, which allows the feasibility of the plan to be re-established while ensuring that all the constraints of the problem are met. This algorithm is denoted as *Reallocate Linked Visits Algorithm*.

t_move_k* and *t_swap_k

These two neighborhoods provide for the movement of one or two customers between different vehicles k, keeping the day of visit unchanged t. Since the day remains fixed, the visit combination associated with the client is not altered and therefore there is no violation of the specific constraints of the PVRP relating to visit patterns.

In the *move*, the customer to be relocated is randomly selected within a route defined by the vehicle–day pair. The new destination route is chosen from those belonging to the same day, but referring to vehicles other than the one of origin.

Similarly, the *swap* operation selects two customers belonging to the same day but assigned to different vehicles, exchanging their positions between their respective routes. Insertion into the new location within the route is done via the proximity-based mechanism described above.

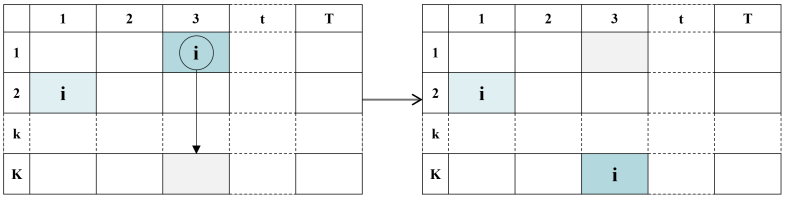


Figure 4.1: *t_move_k*

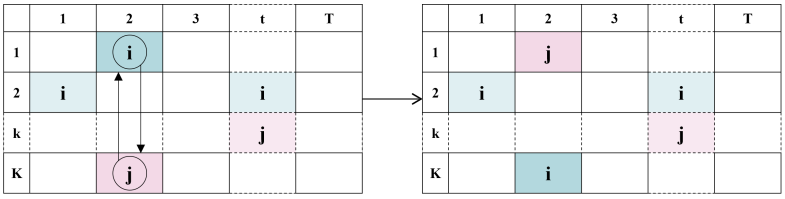


Figure 4.2: *t_swap_k*

move_t* and *swap_t

In this group of neighbors, the customer's transfer necessarily takes place on a different visit day t than the initial one, while the destination vehicle k can be either an existing one or a new vehicle activated for that day. As the day of assignment changes, the move changes the client's time configuration and thus requires explicit control to ensure compatibility with the visit patterns predicted by the PVRP. Should the operation result in a violation of the constraints on customer visit combinations, the dedicated visit reallocation algorithm is activated, in order to restore compliance with the PVRP constraints.

In the case of the *move* operation, a customer is pulled from a route, identified by a day–vehicle pair, and relocated to a route belonging to a different day.

The *swap* transaction, on the other hand, takes into account two customers placed on separate days and exchanges their time and vehicle assignments with each other.

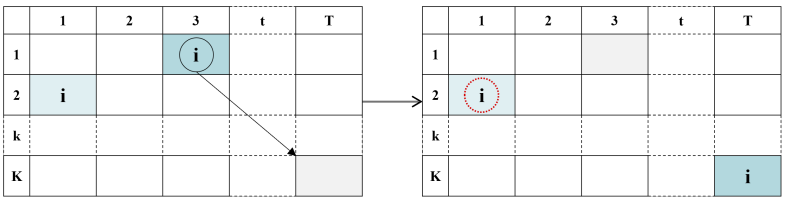


Figure 4.3: *move_t*
The dashed red circle indicates a violation of the constraints on visit combinations.

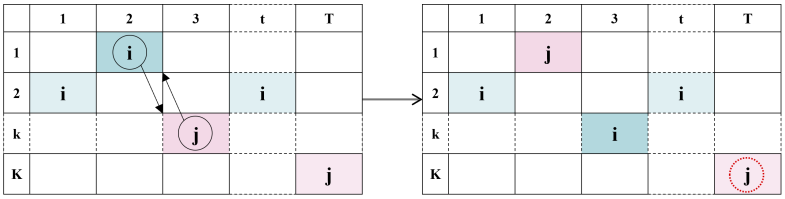


Figure 4.4: *swap_t*
The dashed red circle indicates a violation of the constraints on visit combinations.

f_swap_t

This neighborhood has a structure similar to *swap_t*, but is distinguished by the criterion for selecting the customers involved: the two customers are in fact chosen so that they share the same frequency of visits. Consequently, the exchange does not only concern their main assignments, but all day–visit combinations associated with their respective patterns. In practice, the two clients exchange their visit patterns in full. This mechanism automatically ensures compliance with the constraints relating to visit combinations.

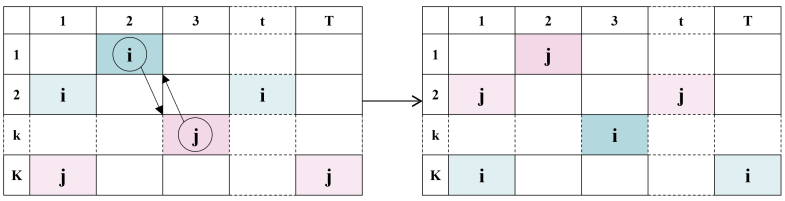


Figure 4.5: *f_swap_t*

t_multi_swap_k

The *t_multi_swap_k* neighborhood selects a group of customers within a randomly chosen route, defined by the vehicle k and the day t, and a second group of similar size (1 customer) from a route belonging to the same day t but to a different vehicle k', so as to have a comparable number of nodes. Before switching between the two routes, the capacity available on both vehicles is verified: only if there is sufficient space is the operation performed, thus ensuring feasibility with respect to capacity constraints.

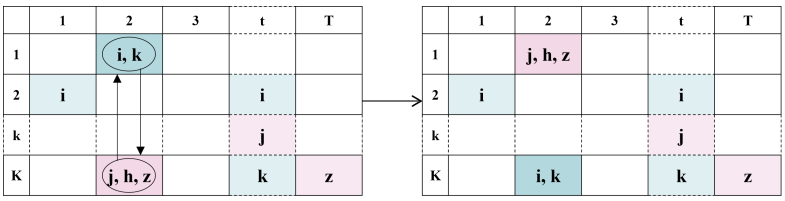


Figure 4.6: *t_multi_swap_k*

t_route_swap_k

In this swap variant, two routes belonging to the same day are randomly exchanged between different vehicles. This neighborhood is designed to act solely on similarity, as assigned customers and their order within routes remain unchanged. The goal of route exchange is to increase the internal consistency of the solution, reducing the total number of customers visited by each vehicle over the entire period. It is hypothesized that an increase in internal similarity could favor an improvement in external similarity, that is, the comparison between different solutions, with particular reference to *nodes-set consistency*.

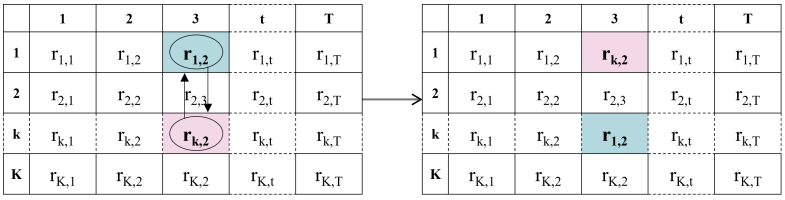


Figure 4.7: *t_route_swap_k*

2-opt* and *3-opt

2-opt and *3-opt* neighborhoods are classical local optimization operators, aimed at improving the quality of solutions through internal modifications to the single route.

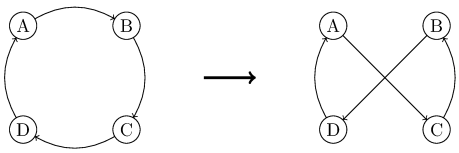


Figure 4.8: *2-opt* [54]

The *2-opt* neighborhood consists of selecting two arcs within the same route and replacing them with two new arcs in order to reverse the customer segment between the selected extremes. This mechanism allows you to reduce the total length of the route by eliminating any intersections between routes, without modifying the customers visited.

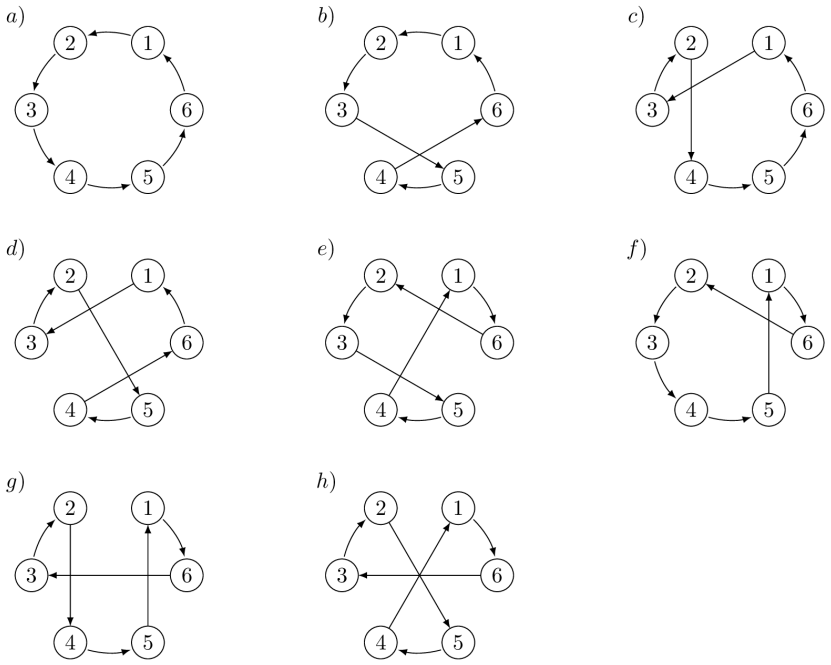


Figure 4.9: *3-opt* [54]

The *3-opt* neighborhood extends the *2-opt* principle by selecting three edges along the route and considering all possible connections of the separate segments. This operation increases the flexibility of local search and allows you to explore a wider range of improvements than the *2-opt*, while remaining within the same route and without changing the set of assigned customers.

Reallocate Linked Visits Algorithm

In the context of a main operation, such as a *move_t* or a *swap_t*, the affected customer can be moved on a different day than its original assignment. This means that new allocations may not meet the constraints of the visit combinations expected for that customer. To handle this situation, the algorithm takes into account all assignments related to the main operation, called *linked combinations*, and proceeds to a customer reallocation to satisfy both vehicle capacity constraints and possible visit combinations.

The process, defined in algorithm 4, begins by defining, starting from the client's main combination, all possible combinations of visit days that can be adopted. Subsequently, these combinations are filtered based on the available capacity of the vehicles, so as to retain only the feasible ones. Once the eligible combinations have been identified, a valid combination is randomly selected for each day the customer is to be visited, with the exception of the main day, which remains fixed. Finally, the customer is assigned to the updated solution and vehicle routes are re-optimized taking into account distance and proximity between customers, so as to consistently integrate the new assignments without violating the capacity constraints or visit combinations expected from the problem. This step ensures that, after the main operation, the resulting solution remains feasible and consistent with the PVRP rules.

Algorithm 4: Reallocate Linked Visits Algorithm

Input: Data about visits, vehicles, and day availabilities
Output: Updated solution or failure

```
1 Evaluate all possible visit combinations;
2 Check feasibility for all vehicle-day routes and update availabilities;
3 if at least one visit combination is satisfied by the availabilities then
4 |   Assign the visits and update the solution;
5 end
6 else
7 |   Return failure (no new solution possible);
8 end
```

4.2.3 Adaptive Variable Neighborhood Search Algorithm

In the *Adaptive Variable Neighborhood Search Algorithm* (Algorithm 5) the selection of neighborhood structures no longer follows a deterministic, sequential or hierarchical scheme. Conversely, neighborhoods are chosen randomly, according to probabilities that they dynamically update over the course of the algorithm. These probabilities are not fixed, but evolve as a function of

the quality of the solutions generated: neighborhoods that demonstrate greater effectiveness are progressively favored, while those that perform less well tend to be selected less frequently.

The mechanism underlying this dynamics is the reward-based weighting system (*reward-based weighting system*), adopted to reward the virtuous behavior of neighborhood and guide research towards promising regions of the solution space. In this study, rewards are structured into four levels, each associated with the goodness of the new x^t solution produced: - w_1: attributed when x^t constitutes a new *global best solution* x^b;
- w_2: assigned when x^t improves the *current solution* x^c;
- w_3: applied when x^t is pejorative with respect to x^c, but is still accepted thanks to the eligibility and tolerance criteria;
- w_4: Assigned when x^t is rejected, typically due to non-feasibility or excessive worsening of the objective value.

The choice of the rewards scale is a particularly relevant element, since it determines the sensitivity of the system in distinguishing between different performances. A scale that is too flat, such as $(4, 3, 2, 1)$, may not be able to highlight substantial differences between neighborhoods, leading to a probability distribution that remains nearly uniform and therefore poorly adaptive. Conversely, more pronounced or exponential scales, such as $(8, 4, 2, 1)$, introduce a *marked divergence* between the most effective and the least productive neighborhoods, allowing for a more targeted and responsive selection. Ultimately, defining the rewards scale is not a marginal choice: it profoundly influences the exploratory behavior of the algorithm, its adaptability, and, ultimately, the overall effectiveness of AVNS in converging on high-quality solutions.

The operational structure of the algorithm first involves selecting a neighborhood randomly, based on the probabilities associated with each of them. Once selected, the neighborhood is executed for a maximum number of iterations defined by the *maximum iterations for neighborhood* parameter. Starting from the current solution x^c, the neighborhood structure iteratively generates new temporary solutions x^t, updating x^c and the global best solution x^b if the new solution has a better objective value. An essential aspect is that, if x^t turns out to be a new global best solution, the A-VNS immediately stops exploring the neighborhood and returns to the selection phase; otherwise, the search continues until the maximum expected number of iterations is completed. A further fundamental parameter is *maximum iterations without improvement*, which establishes the number of consecutive iterations without improvement of the current solution beyond which the algorithm is allowed to accept a pejorative solution. To be accepted, this solution must respect a maximum limit of deterioration of the objective value with respect to x^c, set by the parameter *worsening percentage*. If the solution x^t falls within that margin, it replaces x^c, allowing the algorithm to explore potentially promising new regions.

Algorithm 5: Adaptive Variable Neighborhood Search (A-VNS)

Input: Initial solution x^c, neighborhood set $\mathcal{N}$, time limits, reward parameters
Output: Best solution x^b

Initialize $x^b \leftarrow x^c$, timers, rewards $\overline{X_i}$ and probabilities $P(i)$;
while *elapsed time* $<$ *time limit A-VNS* **do**
  if *time since last global improvement* $>$ *time limit with no improvement* **then**
    Terminate early;
  end
  Choose neighborhood $N_i \in \mathcal{N}$ according to probabilities $P(i)$;
  Apply mandatory route improvement (2-opt and 3-opt) to x^c;
  Initialize $W_i \leftarrow 0$ and $N_i \leftarrow 0$;
  while $N_i <$ *maximum iterations for neighborhood* **do**
    Generate temporary solution x^t from x^c using N_i;
    if x^t *is infeasible* **then**
      Add rejection reward w_4 to W_i and continue;
    end
    if $f(x^t) < f(x^b)$ **then**
      $x^b \leftarrow x^t$, $x^c \leftarrow x^t$;
      Add best-solution reward w_1 to W_i;
      Reset improvement timer;
      break neighborhood loop;
    end
    else if $f(x^t) < f(x^c)$ **then**
      $x^c \leftarrow x^t$;
      Add improvement reward w_2 to W_i;
    end
    else if *iterations without improvement exceed threshold* **then**
      Compute acceptance threshold $f_{\max}$;
      if $f(x^t) < f_{\max}$ **then**
        Accept worsening solution: $x^c \leftarrow x^t$;
        Add acceptance reward w_3 to W_i;
      end
    end
    else
      Add rejection reward w_4 to W_i;
    end
    Increment N_i;
  end
  UpdateRewards(X_i, W, N_i);
  UpdateProbabilities(X, S);
end
return x^b;

Once the neighborhood execution is complete, the rewards are updated according to the following *learning policy*:

$$\overline{X_i} = \overline{X_i} + \frac{W - \overline{X_i}}{N_i}, \tag{4.1}$$

where $\overline{X_i}$ represents the average value of the rewards obtained from the neighborhood i, W is the sum of the rewards observed during the run, and N_i indicates the number of iterations performed. Once the average rewards are recalculated, the neighborhood selection probabilities are updated via:

$$P(i) = \frac{\overline{X_i}}{\sum_i \overline{X_i}}. \tag{4.2}$$

Based on these probabilities, a new neighborhood is then selected, and the cycle continues until the *time limit A-VNS* is reached. An additional time constraint, called *time limit with no improvement*, is also provided, which prematurely stops the algorithm if a new global best solution is not identified within a fixed time window.

The set of neighborhoods used in the A-VNS is as described in Paragraph 4.2.2. It should be noted, however, that not all structures follow the rewards mechanism. Whenever a customer is added or removed from a route, the route to serve all nodes is recalculated using a proximity criterion, producing a reasonably good but not necessarily excellent solution. To prevent the algorithm from perpetuating approximate solutions, the operations of *2-opt* and *3-opt* are always automatically applied to every change in neighborhood. Therefore, while the variants of *move* and *swap* are selected probabilistically according to the accumulated rewards, the neighborhood-houses of type *2-opt* and *3-opt* are forcibly applied before each new selection, ensuring constant refinement of the routes and higher quality of the generated solutions.

4.3 Summary: approach to the complete problem

The Algorithm 6 formally describes the main procedure adopted to address the entire problem, including the generation of reference and perturbed instances, their optimization, and similarity evaluation.

For each p instance of the test set, the algorithm proceeds according to the following main steps:

1. **Preparing and optimizing the reference instance.**
   The data from the original instance is initially processed using the PrepareData function. Next, an initial solution is constructed with AssignmentAlgorithm. If this solution is weak, it is corrected using CompletionAlgorithm. The solution thus obtained is further

Algorithm 6: Main Procedure to the Complete Problem

Input: Set of instances, AVND parameters, similarity measures, demand fluctuation parameters
Output: Solutions, similarities, multi-objective optimized results

```
1  foreach instance p do
       // --- REFERENCE INSTANCE ---
2      reference_data ← PrepareData(reference_instance : p);
3      reference_initial_solution ← AssignmentAlgorithm(reference_data);
4      if reference_initial_solution is infeasible then
5          reference_initial_solution CompletionAlgorithm(reference_initial_solution);
6      end
7      reference_primary_solution ←
        VRP_OptimizationAlgorithm(reference_initial_solution);
       // --- PERTURBED INSTANCE (Demand Fluctuation) ---
8      perturbed_instance ← CreatePerturbedInstance(reference_data, fluctuation
        parameters);
9      perturbed_data ← PrepareData(perturbed_instance : p′);
10     perturbed_initial_solution ← AssignmentAlgorithm(sorted_data_new);
11     if perturbed_initial_solution is infeasible then
12         perturbed_initial_solution CompletionAlgorithm(perturbed_initial_solution);
13     end
14     reference_primary_solution ←
        VRP_OptimizationAlgorithm(perturbed_initial_solution);
       // --- SIMILARITY BETWEEN BASE AND NEW ---
15     ComputeSimilarity(reference_primary_solution, perturbed_primary_solution);
       // --- MULTI-OBJECTIVE A-VND OPTIMIZATION ---
16     foreach similarity measure sim_i ∈ {sim_1, sim_2, sim_3} do
17         current_multiOBJ_sol, best_multiOBJ_sol ← perturbed_primary_solution
18         perturbed_refined_solution ← MultiObjAVND(reference_primary_solution,
            current_multiOBJ_sol, best_multiOBJ_sol, neighborhoods, A-VND parameters,
            similarity measure sim_i);
           // --- SIMILARITY BETWEEN BASE AND OPTIMIZED NEW ---
19         ComputeSimilarity(reference_primary_solution, perturbed_refined_solution);
20     end
21 end
```

refined through the VRP_OptimizationAlgorithm, generating the *reference primary solution*, which constitutes the reference point for the subsequent steps.

2. **Generation and optimization of the perturbed instance.**
   The perturbed instance is created by applying demand fluctuations to the reference solution via CreatePerturbedInstance. The data from the new instance is processed with PrepareData, then an initial solution is constructed with AssignmentAlgorithm and corrected if necessary via CompletionAlgorithm. The perturbed solution is finally optimized with VRP_OptimizationAlgorithm, producing the *perturbed primary solution*.

3. **Calculating the similarity between the reference solution and the perturbed solution.**
   Similarity measures between the reference and perturbed solutions are calculated using ComputeSimilarity.

4. **Multi-objective optimization via A-VND.**
   For each similarity measure $sim_i \in \{\text{sim}_1, \text{sim}_2, \text{sim}_3\}$, the MultiObjAVND procedure is started, which takes as input the reference primary solution, the current solution and the current best solution, the available neighborhood structures, the parameters of the A-VND algorithm and the considered similarity measure. The procedure generates the optimized *perturbed refined solution* in multi-objective optics.

5. **Calculating the similarity between the reference solution and the refined solution.**
   At the end of the optimization, the similarity between the reference solution and the refined solution is calculated again using ComputeSimilarity, allowing us to evaluate the effectiveness of the optimization with respect to the selected consistency measure.

The complete execution of the algorithm produces, for each instance and for each similarity measure, both the optimized solutions and the related similarity evaluations before and after the application of A-VND, providing a complete picture of the algorithm's performance in different multi-objective scenarios.

4.3.1 Experiments and implementation details

The experiments were executed on a node equipped with a 2.3 GHz Intel Xeon Gold 5118 processor. The code was developed using Python 3.10.18.
Table 4.1 reports the number of iterations used to solve the VRP Optimization Algorithm via the *pyvrp* package, together with all parameters adopted in the simulations.

Parameter	**Value**
VRP Optimization Algorithm	
Max VRP iterations	500
Demand fluctuation	
Demand fluctuation mode	constant
Fluctuation type	compensated
Shape parameter	2
Weight-based scenarios	
Weights	[0.75, 0.25], [0.5, 0.5], [0.25, 0.75], [0, 1]
Consistency objective measures	
Similarity measures	sim_1, sim_2, sim_3
A-VNS Algorithm	
Time limit VND	1.5 h (5400 s)
Max no improvement time	1 h (3600 s)
Max neighborhood iterations	100
Worse iterations without improvement	20
Worse solution acceptance percentage	0.15
Neighborhoods	t_move_k, move_t, t_swap_k, f_swap_t, swap_t, t_multi_swap_k, t_route_swap_k
Initial score	3
Rewards values	8, 4, 2, 1

Table 4.1: Experimental parameters and scenarios of this study.

Demand fluctuations were generated using a Gamma distribution with shape parameter 2, ensuring that the total demand remained constant; consequently, customer demands varied in a compensated manner. For each weight scenario, three simulations were performed - one for each consistency measure - resulting in a total of 12 simulations across the four weight-variation scenarios.

Regarding the A-VNS algorithm, each simulation was allowed to run for up to 1.5 hours, but it was terminated early if no improvement was observed for one hour. Each neighborhood attempted to find improving solutions for up to 100 iterations; if no improvement occurred, a worsening solution could be accepted after 20 unsuccessful iterations. The neighborhoods considered were *t_move_k*, *move_t*, *t_swap_k*, *f_swap_t*, *swap_t*, *t_multi_swap_k*, and *t_route_swap_k*. Each neighborhood started with an initial score of 3, which was then updated over time according to the reward values 8, 4, 2, and 1.

Chapter 5

Results

This section presents and discusses the results obtained as part of the study, with the aim of providing a comprehensive view of the performance of the adopted methodologies. The data are organized progressively, starting from the analysis of the primary solutions obtained through consolidated approaches (paragraph 5.1), and then deepening the effectiveness of the different neighborhoods used in local optimization (paragraph 5.2). Subsequently, the focus shifts to the results resulting from the application of the multi-objective A-VNS algorithm, with the analysis of the improvements obtained on the objectives and the consistency between the generated solutions (paragraphs 5.3.1 and 5.3.2). Finally, a detailed analysis of the routes on an instance-by-instance basis is presented, highlighting the variations in paths and similarity values as a function of the different scenarios considered (paragraph 5.3.3). The approach followed allows us to evaluate both the absolute performance of the methodologies adopted and the indirect effects of optimization on the various solution quality indicators.

It is important to make known that for some scenarios, optimization aimed at determining refined solutions of specific instances did not produce results. The cause of this failure to complete is unclear: solutions regularly enter the A-VNS algorithm, which has a time limit, but subsequently the process appears to fail to progress, remaining "stuck" without completing the optimization. This anomalous behavior, which could suggest a dead-end internal loop, is unexpected. Instances for which no solution was obtained are designated NS (not solved). In particular, instances 14, 15, and 16 are not solved in the 25–75 scenario, while instances 2, 10, 14, 15, 16, 24, 25, and 26 are not solved in the 0–100 scenario. The total number of instances actually solved for each scenario is reported in Table 5.3.

5.1 Primary solution results

Table 5.1 presents the computational results obtained by Gulczynski et al. (2011) [55], obtained by their new heuristic for the Periodic Vehicle Routing Problem (PVRP), comparing them with the results provided by four other algorithms widely recognized in the literature. All algorithms were applied to a set of benchmark instances, the same ones adopted in this study. The authors, the algorithm used to address PVRP, and the hardware used for each of the five approaches considered are described below.

IPH: The authors are Damon Gulczynski, Bruce Golden and Edward Wasil [55]. The acronym IPH identifies their proposed algorithm, Integer Programming-based Heuristic, which combines an entire improvement program (IIP) with the Enhanced Record-to-Record Travel (ERTR) algorithm. The hardware used consists of a 3.0 GHz Pentium 4 processor with 512 MB of RAM, while the software used includes ILOG CPLEX 10.0 for solving integer programs and Visual C++ (version 6.0).

CGW: The acronym CGW refers to the authors I.-M. Chao, B. Golden and E. Wasil [56]. The algorithm considered is divided into two phases and is based on the Record-to-Record Travel (RTR) procedure for solving the PVRP. The hardware indicated in the publication is a Sun 4/370 workstation.

CGL: The authors J.-F. Cordeau, M. Gendreau, and G. Laporte developed a Tabu Search-type heuristic designed for periodic and multi-depot vehicle routing problems [39]. The declared hardware consists of a 3.2 GHz Pentium 3 processor.

ALP: J. Alegre, M. Laguna, and J. Pacheco addressed PVRP using a Scatter Search algorithm, specifically designed for instances with an extended time horizon [57]. The hardware used is a 600 MHz Pentium 3 processor.

HDH: Finally, V. Hemmelmayr, K. Doerner, and R. Hartl are the authors of the Variable Neighborhood Search (VNS) heuristic developed for periodic routing problems [58]. The hardware used is a 3.2 GHz Pentium 3 processor.

Study results: Table 5.2 compares the results obtained in this study with the best values reported in Table 5.1, indicated as Best Known Results (BKR). Table 5.2 presents both the

Instance	n	m	t	IPH	CGW	CGL	ALP	HDH
p01	50	3	2	**524.61**	**524.6**	**524.61**	531.02	**524.61**
p02	50	3	5	1335.08	1337.2	1330.09	**1324.74**	1332.01
p03	50	1	5	**524.61**	**524.6**	**524.61**	537.37	528.97
p04	75	5	2	838.47	860.9	**837.94**	845.97	847.48
p05	75	6	5	2052.81	2089.0	2061.36	**2043.75**	2059.74
p06	75	1	10	**838.47**	881.1	840.30	840.10	884.69
p07	100	4	2	**827.39**	832.0	829.37	829.65	829.92
p08	100	5	5	2054.25	2075.1	2054.90	**2052.21**	2058.36
p09	100	1	8	**827.39**	829.9	829.45	829.65	834.92
p10	100	4	5	1645.42	1633.2	1629.96	**1621.21**	1629.76
p11	131	4	5	**781.68**	791.3	817.56	782.17	791.18
p12	163	3	5	**1225.78**	1237.4	1239.58	1230.95	1258.46
p13	417	9	7	3624.77	3629.8	**3602.76**	NS	3835.90
p14	20	2	4	**954.81**	**954.8**	**954.81**	**954.81**	**954.81**
p15	38	2	4	**1862.63**	**1862.6**	**1862.63**	**1862.63**	**1862.63**
p16	56	2	4	**2875.10**	2875.2	2875.24	2875.24	2875.24
p17	40	4	4	**1597.66**	1614.4	1597.75	1597.75	1601.75
p18	76	4	4	3215.43	3217.7	3159.22	3157.00	**3147.91**
p19	112	4	4	**4845.97**	4846.5	4902.64	4846.49	4851.41
p20	184	4	4	8369.72	**8367.4**	**8367.40**	8412.02	**8367.40**
p21	60	6	4	2189.00	2216.1	2184.04	**2173.58**	2180.33
p22	114	6	4	4317.17	4436.4	4307.19	4330.59	**4218.46**
p23	168	6	4	6685.07	6769.0	**6620.50**	6813.45	6644.93
p24	51	3	6	3714.98	3773.0	3704.11	**3702.02**	3704.60
p25	51	3	6	3817.08	3826.0	**3781.38**	**3781.38**	**3781.38**
p26	51	3	6	**3794.95**	3834.0	3795.32	3795.33	3795.32
p27	102	6	6	**21946.89**	23401.6	23017.45	22561.33	22153.31
p28	102	6	6	**22384.04**	23105.1	22569.40	22562.44	22418.52
p29	102	6	6	**22628.95**	24248.2	24012.92	23752.15	22864.23
p30	153	9	6	**75003.85**	80982.1	77179.33	76793.99	75579.23
p31	153	9	6	**76842.26**	80279.1	79382.35	77944.79	77459.14
p32	153	9	6	**78650.81**	83838.7	80908.95	81055.52	79487.97
Average run time [s]				621.64	1229	139.2	1449	148.6

Table 5.1: PVRP instances results from five algorithms.
Bold indicates the best solution among the five algorithms. NS – Not solved.

initial results, i.e. the distance values obtained using the *Assignment Algorithm* alone and, in cases where some customers were not assigned, using the *Completion Algorithm*, and the primary results, obtained by also applying daily re-optimization using the *VRP Optimization Algorithm*.

Instance	BKR	Initial solution	Primary solution	Time [s]	Gap from BKR [%]
p01	524.6	705.04	632.80	0.6682	20.63
p02	1324.74	1893.07	1522.63	1.3510	14.94
p03	524.6	705.04	666.23	0.4410	26.99
p04	837.94	1271.05	951.70	1.2528	13.58
p05	2043.75	2924.28	2223.36	2.5207	8.79
p06	838.47	1271.05	1184.55	0.7467	41.28
p07	827.39	1246.29	993.00	2.8680	20.02
p08	2052.21	2869.14	2272.68	4.9554	10.74
p09	827.39	1246.29	1194.05	1.1158	44.31
p10	1621.21	2400.71	1964.20	3.1892	21.16
p11	781.68	1185.11	911.88	5.8366	16.66
p12	1225.78	1733.20	1522.51	11.1765	24.21
p13	3602.76	6510.03	4479.38	37.1449	24.33
p14	954.8	971.93	**954.81**	0.5466	**0.00**
p15	1862.6	2239.86	**1896.11**	0.8660	**1.80**
p16	2875.1	3407.40	3193.77	1.3068	11.08
p17	1597.66	1720.18	**1643.67**	0.9529	**2.88**
p18	3147.91	3425.32	**3215.44**	2.6688	**2.15**
p19	4845.97	5056.36	**4846.49**	4.5837	**0.01**
p20	8367.4	10520.29	9316.93	12.0801	11.35
p21	2173.58	2487.63	**2197.25**	1.7926	**1.09**
p22	4218.46	4770.26	4490.19	5.8331	6.44
p23	6620.5	7372.29	**6824.46**	13.9764	**3.08**
p24	3702.02	5192.80	4232.55	1.1263	14.33
p25	3781.38	5192.80	4382.97	1.4936	15.91
p26	3794.95	5164.46	4481.32	1.5200	18.09
p27	21946.89	34645.59	26246.02	6.8140	19.59
p28	22384.04	34645.59	26849.27	7.5090	19.95
p29	22628.95	34833.25	26881.71	7.1777	18.79
p30	75003.85	127841.67	86704.78	24.2887	15.60
p31	76842.26	130046.55	88645.53	25.0765	15.36
p32	78650.81	128684.64	90639.67	25.8431	15.24
Mean				**6.8351 s**	**15.01 %**

Table 5.2: Instances primary solutions results. Bold indicates solutions with a deviation lower than 5%. BKR – Best Known Results, the best results of Table 5.1.

Overall, as highlighted by the last column, the discrepancy between the primary solutions and the BKRs is high, with an average of 15.01%. However, in seven instances the results obtained show a deviation of less than 5%. These instances are characterized, as indicated in Table 2.1, by an equal number of days equal to 4 and by the same frequency group f_1, f_2 and f_4. These char-

acteristics could be considered to have favored both initial assignment and daily re-optimization. Despite this, instances with the same properties —particularly instances 16, 20, and 22— showed worse performance, with discrepancies ranging from 6.44% to ’11.34%.

The goal of finding the primary solution is essentially to have a valid reference for benchmark instances and a baseline for consistency optimization. For this reason, a rapid and effective approach was adopted, as confirmed by the average times reported in Table 5.2, thus giving less weight to distance optimization. The process, divided into two phases, first involves assigning customers to the routes defined by the vehicle–day combinations, followed by a daily optimization that allows customers to change routes while remaining tied to the same day.

An intrinsic limitation of this approach, due to the nature of PVRP, is the lack of operators capable of moving customers to combinations belonging to different vehicles. It may therefore happen that the initial assignment to vehicles is not optimal and that, due to this structural restriction, it is not possible to modify the assignments between vehicles, thus preventing a further improvement in the target value.

5.2 Neighborhood efficiency results

Neighborhood effectiveness was assessed using four metrics: mean time, mean improvement, total uses, and mean uses. The average time is defined as the time variation between the start and end of neighborhood execution, relative to the total number of iterations performed.

$$mean(time_{neigh}) = \frac{\Delta time}{iterations} \tag{5.1}$$

The improvement was calculated by observing the change in the target value of the current solution, as it is precisely this solution that the algorithm modifies and evaluates in order to identify better solutions. The variation can be negative, since, after a certain number of solutions without improvement, the algorithm can accept worsening solutions. The average improvement therefore corresponds to the ratio between the difference in target value and the number of iterations.

$$mean(improvement_{neigh}) = \frac{\Delta OBJ_{value}}{iterations} \tag{5.2}$$

Given the adaptive nature of the chosen algorithm, the number of times a neighborhood is called may reflect the effectiveness of the neighborhood structure. For this reason, the call counts of each neighborhood were calculated for each instance, denoted as $total(count_{neigh})$. However, considering the value alone can be misleading, as some simulations may not have produced results for all instances, thus lowering the count. For this reason, the average number of calls is also

evaluated, calculated as the ratio between the total count and the number of valid instances.

$$mean(count_{neigh}) = \frac{total(count_{neigh})}{instances_{working}} \tag{5.3}$$

As previously noted, it should be noted that not all simulations were able to produce results for each instance. The number of instances for which a refined solution was obtained in each simulation is presented in the Table 5.3. The reasons behind these shortcomings are discussed in paragraph 5.3.

Simulation weights	Running instances
75-25	32
50-50	32
25-75	29
0-100	24

Table 5.3: Running instances per simulation.

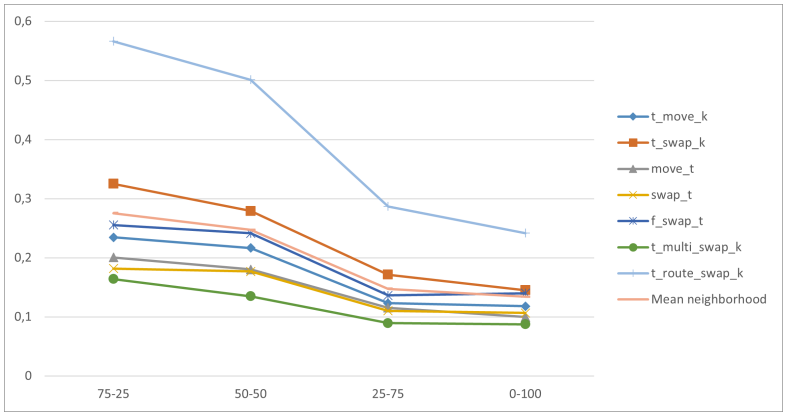
The results on neighborhood efficiency are reported in Table 5.4, where the four metrics are presented for each simulation, along with the total mean for each neighborhood, reported in the last four rows. Additionally, the average neighborhood efficiency for each simulation, shown in the last column on the right, was calculated.

Looking at the average time of each neighborhood, we see that this value decreases as the weight assigned to the consistency increases. As highlighted by the graph in Figure 5.1a, this trend is followed by all neighborhoods, with slight variations. Conversely, the average number of calls for each neighborhood increases as the weight dedicated to consistency increases. This behavior is predictable, since reducing the average time allows for a greater number of neighborhood runs in the same time interval as the algorithm. The trend of the average counts is represented in Figure 5.1b. This is a particularly interesting phenomenon, the interpretation of which is not immediate, since the structure of the neighborhoods remains unchanged as the weights vary and does not introduce changes that directly justify such a marked increase in the number of uses and a decrease in times. The average improvement, however, does not show a clear trend as a function of the weights assigned to the objective function. However, as can be seen more clearly in Figure 5.1c, each neighborhood achieves maximum performance when optimizing similarity alone.

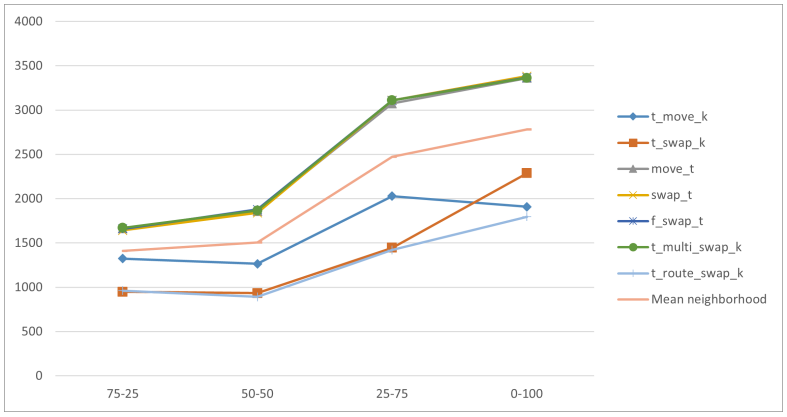
Analyzing the neighborhoods individually, we note that *t_route_swap_k* has the longest times and the lowest number of uses, followed by *t_swap_k* and *t_move_k*. Despite this, *t_swap_k* records high average improvement values. This behavior appears anomalous, since the reward mecha-

		t_move_k	*t_swap_k*	*move_t*	*swap_t*	*f_swap_t*	*t_multi_swap_k*	*t_route_swap_k*	Mean neighborhood
75-25	Mean time	0,234593649	0,325181991	0,200531276	0,181753031	0,255234071	0,164032682	0,566262179	0,27536984
	Mean improvement	-0,086103641	1,225178398	0,624029678	0,873077851	0,679834087	-0,16554159	-0,553758827	0,370959422
	Total count	42338	30322	53289	52553	53102	53442	30791	45119,57143
	Mean total count	1323,0625	947,5625	1665,28125	1642,28125	1659,4375	1670,0625	962,21875	1409,986607
50-50	Mean time	0,216592952	0,279110358	0,180030697	0,176615572	0,241194282	0,134638846	0,501100336	0,247040435
	Mean improvement	-0,003211661	1,038428365	0,257082934	0,629960178	1,074367636	-0,101677661	-0,640189026	0,322108681
	Total count	40498	29848	59455	58844	60121	59752	28516	48147,71429
	Mean total count	1265,5625	932,75	1857,96875	1838,875	1878,78125	1867,25	891,125	1504,616071
25-75	Mean time	0,12299724	0,171422046	0,115255563	0,11027912	0,136147334	0,08929426	0,286656909	0,147436067
	Mean improvement	-0,163519129	1,033889176	0,382426241	0,809845316	0,534651278	-0,235544721	-0,595717113	0,25229015
	Total count	58852	41927	89175	90209	90167	90203	41206	71677
	Mean total count	2029,37931	1445,758621	3075	3110,655172	3109,206897	3110,448276	1420,896552	2471,62069
0-100	Mean time	0,117769028	0,144986875	0,099540984	0,106388082	0,139860968	0,087388695	0,241499961	0,133919228
	Mean improvement	0,363816738	1,691179497	0,599521894	0,742816313	0,847168447	0,45930515	-0,013048939	0,670108443
	Total count	45791	54900	80689	81188	80827	80720	43126	66748,71429
	Mean total count	1907,958333	2287,5	3362,041667	3382,833333	3367,791667	3363,333333	1796,916667	2781,196429
Mean simulation	Mean time	0,172988217	0,230175318	0,14883963	0,143758951	0,193109164	0,118838621	0,398879846	0,200941392
	Mean improvement	0,027745577	1,247168859	0,465765187	0,763924914	0,784005362	-0,010864705	-0,450678476	0,403866674
	Total count	187479	156997	282608	282794	284217	284117	143639	231693
	Mean total count	6525,962644	5613,571121	9960,291667	9974,644756	10015,21731	10011,09411	5071,156968	8167,419797

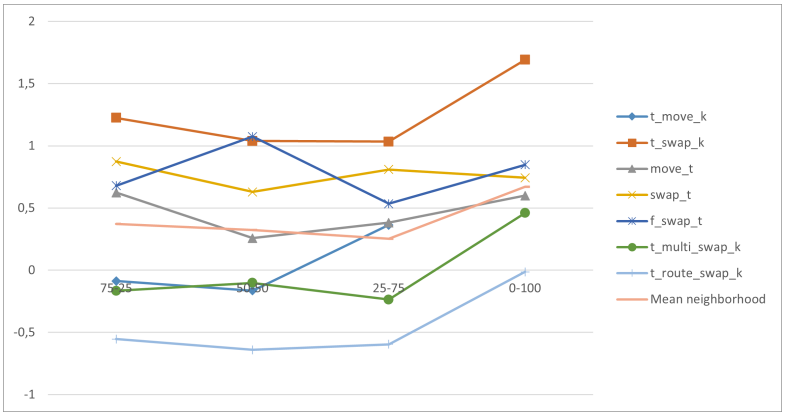
Table 5.4: Neighborhoods efficiencies.



(a) Mean time per neighborhood.



(b) Mean count per neighborhood.



(c) Mean improvement per neighborhood.

Figure 5.1: Efficiency measures per neighborhood.

nism is based precisely on improving the solution: the more a neighborhood is able to improve the solution, the higher its reward. A similar situation is observed for *t_multi_swap_k*, which has a very low average improvement (−0.010864705) but a relatively high number of uses, placing itself in the graph in Figure 5.2 on a par with *move_t*, *swap_t* and *f_swap_t*. These last three neighborhoods show an optimal balance between metrics, with average timing and improvement, but excelling in the number of uses.

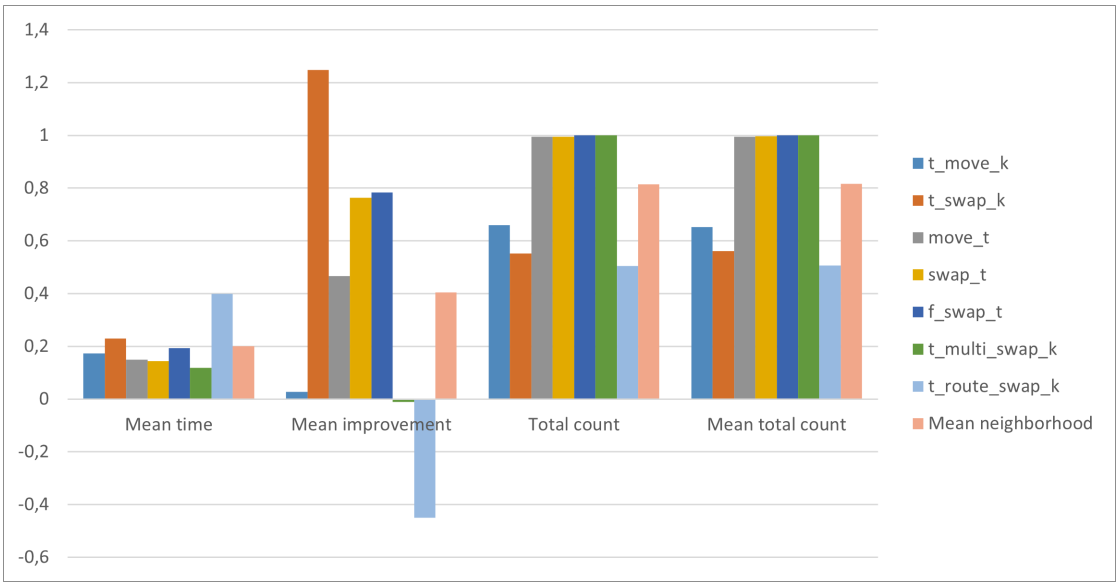


Figure 5.2: Comparison between neighborhood efficiencies.
The "total count" and "mean total count" measures were normalized by their respective maximum values, 284217 and 10015.21731 (both corresponding to the *f_swap_t* neighborhood), in order to ensure a proper fit within the graphical representation.

The worst performing neighborhoods in terms of average improvement are *t_route_swap_k*, followed by *t_multi_swap_k* and *t_move_k*. There can be many causes; in the case of *t_route_swap_k* the behavior is probably due to the fact that this neighborhood was evaluated indifferently by the similarity measure used. As discussed in the 4.2.2 paragraph, this neighborhood was designed primarily in relation to the *nodes-set consistency*. For *t_multi_swap_k*, the multiple swap randomly selects a sequence of customers, some of whom improve and some of whom worsen distance and/or similarity. Despite this, the neighborhood remains widely used, suggesting that it is often chosen when running the algorithm. To improve efficiency, customers could be selected not necessarily sequentially, but based on criteria such as distance from the route cluster. Finally, *t_move_k* may have difficulty improving the solution, since it is a more limited move, acting within the same day already optimized by the *VRP Optimization Algorithm*.

If the VND, with a fixed and time-efficient neighborhood structure, had been adopted, the or-

der would have been: *t_multi_swap_k*, *swap_t*, *move_t*, *t_move_k*, *f_swap_t*, *t_swap_k* and, finally, *t_route_swap_k*. However, analysis of the different metrics revealed discrepancies, and establishing a fixed hierarchy could reduce the efficiency of the algorithm. Regarding the adaptive reward-based method, some weight choices may seem questionable, as some low-improvement neighborhoods are still widely used. In general, all neighborhoods showed average or high performance, with the exception of *t_route_swap_k*, which may still be useful for avoiding local minima.

2-opt and 3-opt

The neighborhood *2-opt* and *3-opt* were applied to all instances starting from the initial solution generated by the *Assignment Algorithm*. In particular, they were tested both individually and sequentially: the *2-opt*, the *3-opt*, the *2-opt* followed by the *3-opt* (called *2-3-opt*) and finally the *3-opt* followed by the *2-opt* (named *3-2-opt*).

For each of these variants, the number of instances in which it was able to improve the starting solution was counted. In order to compare them and identify the most effective, in each test the best solution obtained among the four received a point in the *rank_1* column; in case of lower performance, the point was assigned to the *rank_2*, *rank_3* or *rank_4* columns, while no points were assigned if no improvement was achieved. The results are reported in Table 5.5.

From the observed values, the *2-3-opt* method is the one with the best overall performance, in terms of the total number of improvement solutions. Next comes the *3-2-opt*, with only one point difference in the overall count.

	rank_1	rank_2	rank_3	rank_4	sum
2-opt	350	136	45	11	542
3-opt	57	43	43	35	178
2-3opt	497	48	3	0	548
3-2opt	446	91	10	0	547

Table 5.5: 2-opt and 3-opt results

A more in-depth analysis highlighted that, of the 719 instances where the two combined methods were applied, in 464 cases they produced the same solution. This means that l'84.83% of the solutions found by *3-2-opt* coincide with those obtained by *2-3-opt*, and vice versa. However, the *2-3-opt* method has an additional advantage: it is able to identify better solutions in a higher number of cases, as shown by the values in the *rank_1* column. A similar observation emerges when comparing *2-opt* and *3-opt* taken individually.

Furthermore, the adoption of the *2-3-opt* method brings an additional operational benefit: since it applies *2-opt* first and *3-opt* subsequently. Running *2-3-opt* also implicitly allows the calculation

of *2-opt* alone, which gets a very high value in both the *rank_1* and the total count . For these reasons, within the A-VND algorithm it was chosen to adopt the neighborhood *2-3-opt*.

5.3 Multi-objective algorithm results

This section is dedicated to the analysis of the results obtained using the A-VNS multi-objective optimization algorithm, applied to the primary solutions of the perturbed instances. The aim is to evaluate the effectiveness of the procedure in improving the values of the objectives considered, as well as to understand its impact on the consistency and overall quality of the generated solutions. The discussion is structured by considering different scenarios characterized by different weights between the objectives, in order to highlight the improvement dynamics and possible interactions between similarity metrics. This approach allows for a thorough understanding of the algorithm's potential, as well as the trade-offs involved in simultaneously managing multiple optimization criteria.

5.3.1 Evaluation of objective improvement

This section aims to discuss the results obtained from the refined solutions generated via the A-VNS optimization algorithm, starting from the primary solutions of the perturbed instances. Four simulation scenarios characterized by different weights were considered: 75-25, 50-50, 25-75 and 0-100, while the 100-0 scenario corresponds to the primary solution of perturbed instances.

The results related to the individual scenarios are reported in Table 5.6. The Table reports the mean values from the results obtained from the optimization for each instance, where the data is organized as follows:
- *P OBJ*: primary objective value;
- *R OBJ*: refined objective value;
- *P distance*: primary distance value;
- *R distance*: refined distance value;
- *P* sim_i: primary consistency value for similarity i;
- *R* sim_1: refined consistency value for similarity 1;
- *R* sim_2: refined consistency value for similarity 2;
- *R* sim_3: refined consistency value for similarity 3;
- *time*: running time of the algorithm.

Furthermore, the percentage improvement values of both the target value (*objective value*) and the distance, as well as the difference between the consistency measures, were calculated. In

particular, the percentage improvement of the target value is defined as:

$$improve_{OBJ}[\%] = \left(\frac{R\,OBJ}{P\,OBJ} - 1\right) \cdot 100 \tag{5.4}$$

This metric allows us to quantify the overall effectiveness of the optimization, highlighting any improvements compared to the starting solution.

Similarly, the percentage improvement in distance is calculated as:

$$improve_{dist}[\%] = \left(\frac{R\,dist}{P\,dist} - 1\right) \cdot 100 \tag{5.5}$$

This measure allows us to evaluate how optimization influences route efficiency, considering reductions or increases in distances traveled.

Finally, the difference between consistency measures is defined as:

$$\Delta_{consistency} = R\,sim_i - P\,sim_i \tag{5.6}$$

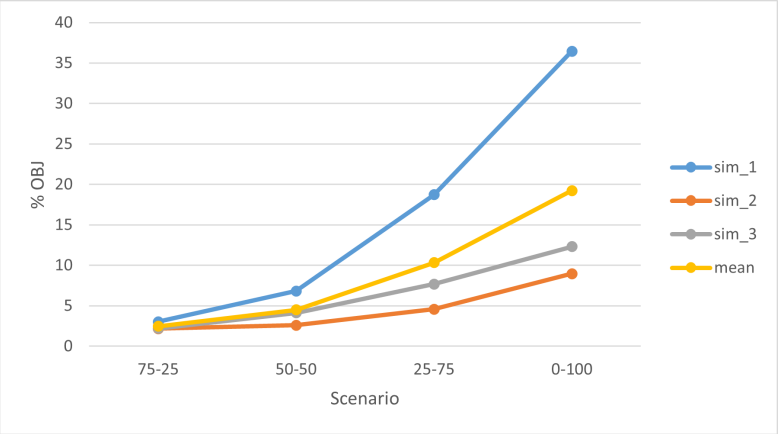
where $R\,sim_i$ and $P\,sim_i$ represent the consistency values of the refined solution and the primary solution according to the measure i, respectively. This indicator is essential for understanding the extent to which optimization increases the consistency of the generated solutions, allowing for the simultaneous balancing of different performance criteria.

Analyzing these metrics under different weight scenarios (75-25, 50-50, 25-75, and 0-100) allows us to understand the relative impact of each criterion on optimization. In particular, scenarios with a higher similarity weight are expected to tend to show more noticeable increases in *consistency*, while scenarios with a predominant distance weight show smaller increases in distance traveled. Similarly, the percentage improvement in the target value synthetically reflects the algorithm's overall ability to generate more efficient solutions than the primary solution, highlighting the areas of the solution space most influenced by the assigned weights.

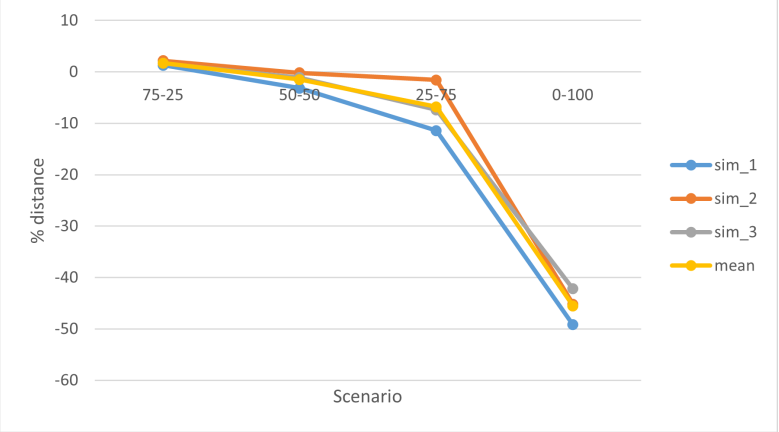
The average values of these quantities, calculated on all instances, are reported in Table 5.6. The column $P\ sim_i$ has been omitted, since its value varies as a function of the consistency measure considered. The quantities are organized and classified based on the similarity measure adopted and the related simulation scenario.
The data in Table 5.6, together with the graphs shown in Figures 5.3a, 5.3b and 5.3c, show the trends of the percentage improvement of the target value, the percentage improvement of the distance and the difference in consistency respectively as a function of the simulation scenarios, considering each of the three similarity measures.

Scenario	Sim	P OBJ	R OBJ	P dist	R dist	Time	OBJ %	Dist %	Δ consistency
75–25	sim_1	0.908383817	0.880843373	12984.35965	12977.56770	3918.26682	3.031806989	1.315056135	0.066309748
	sim_2	1.006393013	0.984436186	12984.35965	12951.60831	3950.973491	2.181734896	2.168487484	0.018253475
	sim_3	1.005418232	0.983392361	12984.35965	12964.14912	3866.664207	2.190717271	1.672198307	0.033826955
	mean	0.973398354	0.949557307	12984.35965	12964.44171	3911.968173	2.468086386	1.718580642	0.039463393
50–50	sim_1	0.805210569	0.750315062	12984.35965	13115.87836	4220.668105	6.817534327	-3.142080784	0.139287581
	sim_2	1.001228961	0.975215435	12984.35965	13007.81546	4064.071049	2.598159560	-0.170562687	0.053094876
	sim_3	0.992489964	0.951653805	12984.35965	13043.14066	3992.708283	4.114516017	-1.084498610	0.078370552
	mean	0.932976498	0.892394767	12984.35965	13055.61149	4092.482479	4.510069968	-1.465714027	0.090251003
25–75	sim_1	0.724566691	0.588930184	14114.34897	15297.72113	4349.336028	18.71967187	-11.40698944	0.155979757
	sim_2	0.997453998	0.951909775	14114.34897	14280.50710	4125.265880	4.566047506	-1.553575714	0.053272701
	sim_3	0.995425075	0.919045841	14114.34897	14353.92447	4219.828563	7.673026924	-7.377708724	0.125914119
	mean	0.905815255	0.819961933	14114.34897	14644.05090	4231.476824	10.31958210	-6.779424626	0.111722192
0–100	sim_1	0.661475795	0.420261372	16370.34871	20481.76199	4366.043093	36.46609967	-49.18499095	0.241214423
	sim_2	0.994742358	0.905669476	16370.34871	19571.16430	4662.153107	8.954367046	-45.19313919	0.089072882
	sim_3	0.991266858	0.869165219	16370.34871	19526.80482	4497.809647	12.31773649	-42.18375861	0.122101639
	mean	0.882495004	0.731698689	16370.34871	19859.91037	4508.668616	19.24606773	-45.52062958	0.150796315

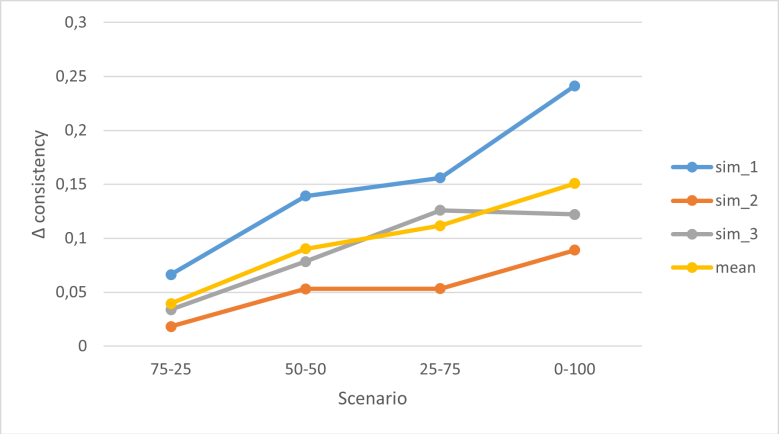
Table 5.6: Simulation results.



(a) Trends of the percentage improvement of the multi-objective value.



(b) Trends of the percentage improvement of the distance.



(c) Trends of the difference in consistency.

Figure 5.3: Trends of the objective values.

The similar trends observed in the curves of all graphs suggest that the algorithm is operating consistently and effectively. In particular, the percentage improvement in the target value shows a slow increase in the first three scenarios, followed by a sudden increase in the simulation with weights between 0 and 100. This peak clearly highlights the significant impact of higher weights on solution improvement. Similarly, the percentage improvement in distance follows the same trend, albeit with negative values. This behavior indicates that the distance tends to worsen, but this is in favor of an increase in similarity between solutions, highlighting the trade-off between the algorithm's objectives. Finally, the consistency difference shows a less marked but constant increase, confirming a positive trend towards greater consistency of the generated solutions. These observations significantly highlight how the algorithm is able to balance multiple optimization criteria in a controlled manner, while maintaining predictable and consistent behavior across simulations.

Looking closely at the distance curves, another notable observation emerges: in the 75-25 scenario, the percentage remains just above 0. This behavior is particularly significant, as it indicates that including consistency in the objective function with a weight of 0.25 not only improves the similarity between solutions, but also helps reduce the distances traveled. This result highlights the effectiveness of the adopted approach, as it allows for optimal balancing of multiple optimization criteria without compromising overall performance.

Analyzing the trend of the curves relating to the different consistency measures, it is immediately observed that *sim_1* turns out to be the most effective measure in guaranteeing superior results. It is important to note that the *node-set consistency*, unlike the other two measures, is calculated by comparing the entire set of customers served by the vehicle, thus providing a more global assessment of the consistency of the solution. The measure *sim_3*, which comes in second place, and finally *sim_2*, instead derive from the average of the similarity values calculated on each route belonging to the vehicle. This approach tends to reduce the perceived effectiveness of the measure, as it fragments the analysis with respect to the entire set of customers. The fact that *arc-set consistency* performs better than *arc-sequence consistency* was predictable, since the former can be considered a generalization of the latter. For example, if two routes under comparison have a total of 7 common edges, of which the maximum contiguous sequence is 4, and the reference route contains 10 edges, then we get $sim_3 = 7/10 = 0.7$, while $sim_2 = 4/10 = 0.4$. This makes it abundantly clear why *sim_3* guarantees more robust and consistent results than *sim_2*.

A further observation concerns the execution times of the algorithm, which show an increase in the different scenarios considered, as can be seen in the graph in Figure 5.4. This behavior is understandable: when the weights are high on distance rather than consistency, the algorithm tends to work on solutions already optimized in the previous stage of the study, using the *VRP*

Optimization Algorithm. Conversely, by increasing the weight associated with similarity, the algorithm is encouraged to explore new areas of the solution space, extending the time required to complete the optimization. It is also important to note that in no scenario do the average times between instances reach the 5400 s expected as the maximum limit for the execution of the algorithm, confirming the overall efficiency of the adopted method.

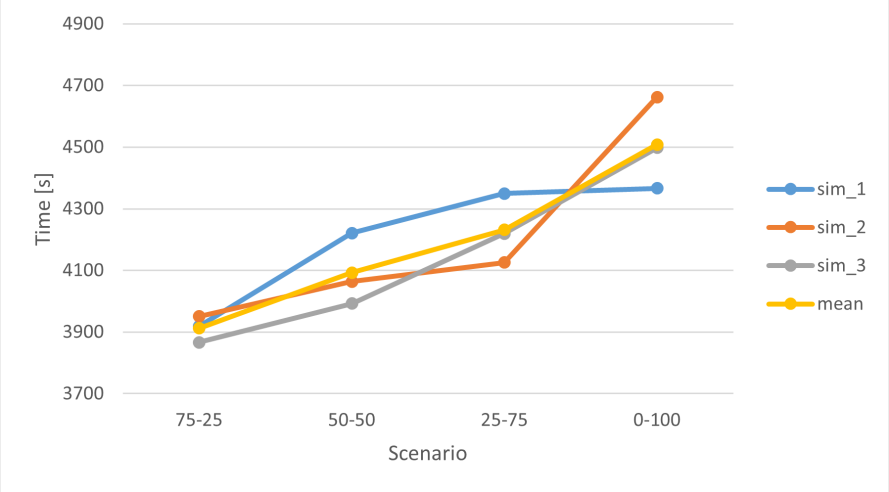


Figure 5.4: Execution times of the algorithm.

5.3.2 Correlation between consistency measures

Based on the refined solutions, the values of all three similarity measures were calculated to obtain a complete picture of the optimization effect. The main objective of this analysis is to verify whether optimization aimed at a specific similarity measure can generate indirect effects on the other two, thus allowing us to identify possible correlations or interferences between the different metrics. This approach allows us not only to understand the effectiveness of the chosen optimization strategy, but also to evaluate the extent to which improving a single measure can translate into an overall benefit for the consistency and quality of the generated solutions. Understanding such interactions is crucial to determining whether it is preferable to optimize a measure individually or whether a more balanced approach that simultaneously considers all similarity metrics is needed.

Table 5.7 reports, for each scenario, the similarity measures considered during the optimization, organized into rows. The first three columns show the similarity values obtained in the refined solutions, while the next three columns highlight the consistency variations of the measures not being optimized. Finally, the last column reports the variation of the actually optimized consis-

tency measure. All presented values are averaged across all working instances, thus providing an overall picture of the impact of optimization on each similarity measure.

The graphs shown in Figure 5.5 show three curves for each graph, each of which represents the variation of a specific similarity measure as a function of its optimization and of the other two measures.

From all three graphs it immediately emerges that *sim_2* and *sim_3* exhibit very similar and closely related behaviors, while *sim_1* behaves in the opposite way.

In the first graph (Fig. 5.5a) the behavior of *sim_1* is analyzed, which shows a marked increase when it itself is optimized. Conversely, optimizing the other two measures has a negative effect on the values of *sim_1*, highlighting a possible interference between the different metrics.

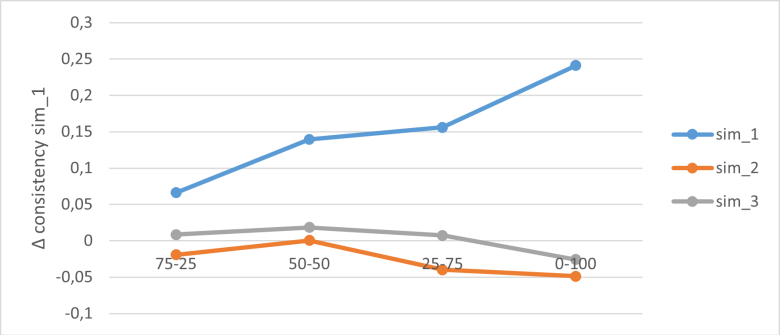
The second graph (Fig. 5.5b) illustrates the variation of *sim_2* as a function of the optimizations of the other measures. For most scenarios, *sim_2* gets the best values when the optimized measure is *sim_3*. Only in the scenario with weights 0-100, optimizing *sim_2* itself allows for maximum improvement. Regarding *arc-sequence consistency*, optimizing *sim_1* results in worse values, confirming the negative effect also observed for *sim_1* in the previous graph.

Finally, the third graph (Fig. 5.5c) once again highlights the close relationship between *sim_2* and *sim_3*. The change in *arc-set consistency* improves significantly thanks to the optimization of *sim_3* and, in particular, of itself, while it undergoes a decrease when the optimization is directed to *sim_1*, confirming the negative trend observed in Fig. 5.5b.

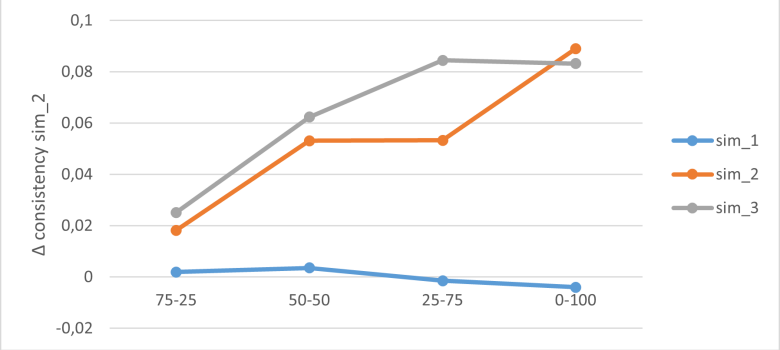
In summary, three similarity measures were evaluated in this study: one aimed at the picking operator (*sim_1*) and two designed to improve path similarity in favor of the driver (*sim_2* and *sim_3*). Given the similar nature of the latter two, similar and closely related behaviors could be observed. However, the opposite behavior of *sim_1* compared to *arc consistencies* was unexpected. In fact, it was hypothesized that all three measures could improve simultaneously and influence each other to a different extent. The results, however, suggest the existence of a negative correlation: improving one measure corresponds to worsening the others and vice versa, indicating the presence of an intrinsic trade-off between the different forms of optimized similarity.

Proportion	Sim	R sim_1	R sim_2	R sim_3	Δ sim_1	Δ sim_2	Δ sim_3	Δ consistency
75-25	sim_1	0.467445674	0.011017603	0.015510011		0.001918459	0.002511742	0.066309748
	sim_2	0.381929472	0.027352619	0.034720124	-0.019206455		0.021721855	0.018253475
	sim_3	0.409850944	0.034253734	0.046825224	0.008715017	0.025154590		0.033826955
50-50	sim_1	0.540423508	0.012625478	0.017150185		0.003526335	0.004151916	0.139287581
	sim_2	0.401781652	0.062194019	0.075131810	0.000645725		0.062133541	0.053094876
	sim_3	0.419481063	0.071505812	0.104947690	0.018345136	0.062406668		0.078370552
25-75	sim_1	0.599461549	0.007626644	0.008489280		-0.001472499	-0.004508989	0.155979757
	sim_2	0.361286515	0.079880285	0.093296425	-0.039849412		0.080298156	0.053272701
	sim_3	0.408492211	0.093602801	0.135495131	0.007356284	0.084503657		0.125914119
0-100	sim_1	0.579738628	0.005150212	0.006954348		-0.003948931	-0.006043921	0.241214423
	sim_2	0.352266622	0.094330524	0.106801972	-0.048869305		0.093803703	0.089072882
	sim_3	0.375300455	0.092272944	0.130834781	-0.025835472	0.083173801		0.122101639

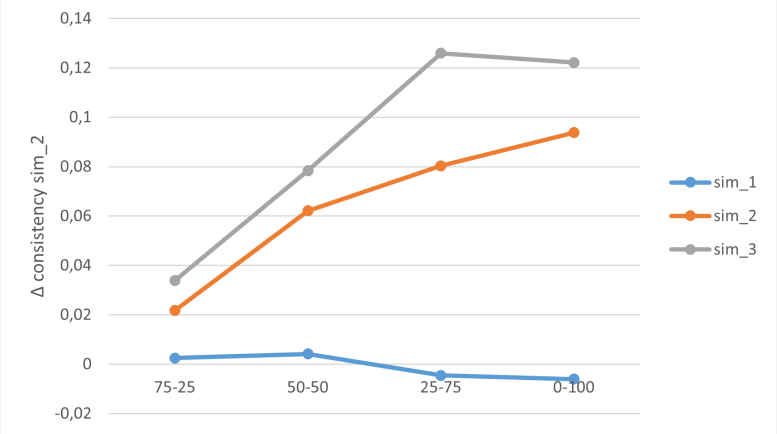
Table 5.7: Correlation of consistency measures.



(a) Trends of *sim_1* as a function of different consistency measures.



(b) Trends of *sim_2* as a function of different consistency measures.



(c) Trends of *sim_1* as a function of different consistency measures.

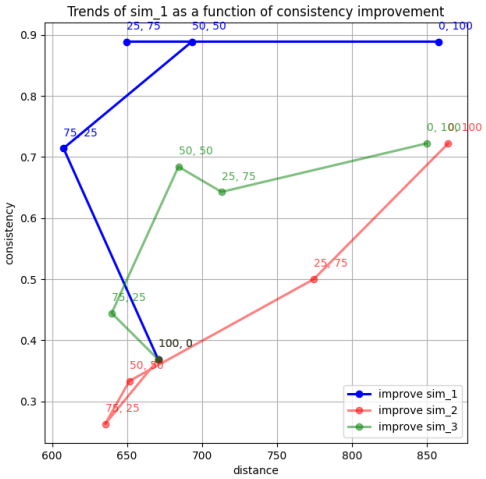
Figure 5.5: Trends of the variations of consistency measures.

5.3.3 Instance routes analysis

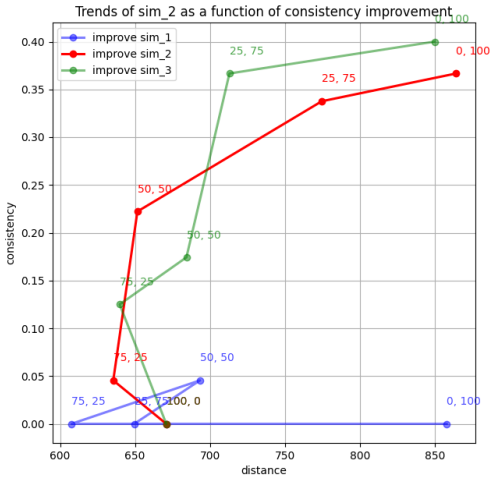
In this section, the routes are analyzed through their graphs (Figure 1, 2 and 3 in Appendix B) and Table 1 (Appendix B), which show how the paths and associated similarity values vary as the optimized consistency measure and the scenario considered change. In particular, Table 1 reports, for each row, the scenario and the consistency measure being optimized; the columns instead describe the couplings between the reference vehicle and the refined solution vehicle, together with the corresponding similarity values according to the three metrics adopted. The highlighted values represent the minimum level of similarity for each measure and scenario, and therefore correspond to the similarity value assumed by the final solution.
To conduct this analysis, the *p01* instance was selected from all thirty-two available instances. This choice was motivated by the good levels of improvement obtained during the optimization phase. All numerical values and graphs discussed below then refer to this specific instance. Figure 5.6 also shows three graphs, one for each similarity measure, which illustrate the trend of the metric as a function of the optimization of the same or of the other two measures.

Looking at the pairings and similarity values presented in Table 1, relating to the optimization of *sim_1*, it emerges that the maximum level of *node-set consistency* is reached already in the 50-50 scenario. In subsequent scenarios, as clearly visible from the plots in Figure 1, the algorithm continues to explore the solution space producing variations in the routes. These changes have no effect on the values of *sim_1*, which remain almost unchanged, while they negatively influence the other two similarity measures, which show a progressive deterioration. The values for *sim_2*, when it is directly optimized, continue to improve as the weight associated with similarity increases, as highlighted by both Table 1 and Figure 5.6b. In parallel, *sim_3* also improves, while *sim_1* initially shows a reduction and then follows, in the subsequent phases, the trend of the other two measures. A qualitatively similar behavior is observed for *sim_3*. Its values increase as one scenario moves to the next, showing trends consistent with increasing similarity weights. Again, optimizing the *arc-set consistency* positively influences *sim_2*, while the effect on *sim_1* is smaller.

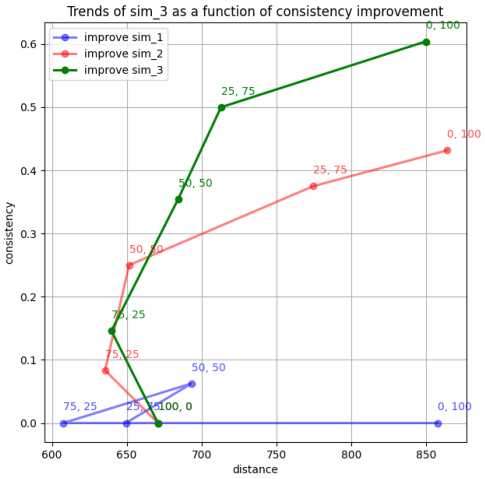
The analysis of the route plots reported in Figures 1, 2 and 3 allows us to formulate a particularly significant visual conclusion. As the weight associated with similarity increases — proceeding from the primary perturbed solution (top right) and scrolling from left to right in the next two lines — the routes take on a shape increasingly similar to that of the reference solution (top left). This evidence confirms, in an intuitive and immediately perceptible way, the effectiveness of consistency optimization in guiding paths towards configurations more consistent with the reference ones.



(a) Trends of *sim_1* for instance *p01*.



(b) Trends of *sim_2* for instance *p01*.



(c) Trends of *sim_3* for instance *p01*.

Figure 5.6: Trends of the variations of consistency measures for instance *p01*.

Conclusions

This study focused on solving the Periodic Vehicle Routing Problem (PVRP) in the context of waste collection (WCVRP) under fluctuating demand conditions. The primary objective was to explore and quantify the trade-off between minimizing total distance traveled (economic efficiency) and maximizing route consistency (route consistency), a crucial factor for operational efficiency and driver and operator satisfaction.

The application of the A-VNS algorithm has demonstrated its effectiveness in balancing the two conflicting objectives. As expected, the optimization showed a clear trade-off: as the weight assigned to consistency increased, the distance traveled tended to worsen (showing a negative percentage improvement), in favor of a significant increase in similarity between solutions. Three different consistency metrics were defined and compared: *Nodes-set* (*sim_1*), *Arc-sequences* (*sim_2*), and *Arcs-set* (*sim_3*). Among these, the *sim_1* measure, which evaluates similarity across the entire set of customers served by vehicles, was found to be the most effective in ensuring superior consistency results. Furthermore, an unexpected negative correlation was observed between *Nodes-set* coherence (*sim_1*) and edge-based coherences (*sim_2* and *sim_3*), suggesting that optimizing one form of coherence can lead to a worsening of the others, defining an additional intrinsic trade-off between the different operational stability metrics. Finally, visual analysis of the routes for the p01 instance confirmed that increasing the consistency weight in the multi-objective leads the refined routes to take a progressively more similar form to that of the reference solution.

Despite promising results, the study has some methodological and operational limitations. The A-VNS algorithm, while effective, failed to resolve all instances, occasionally showing crashes or anomalies during optimization. The number of solved instances decreases with increasing consistency weight, and primary solutions tend to show a significant gap compared to the best known results. Furthermore, some weight choices in local movements (neighborhoods) can be ineffective, and demand modeling keeps the total quantity constant, without reflecting real dynamics such as seasonal, tourist, or extraordinary events.

An important future development of research could, in fact, concern the simulation of more realistic waste fluctuations, reflecting seasonal, tourist variations or extreme events. For example, during the summer, residential areas can empty, reducing waste generation, while tourist areas, hotels, and hospitality businesses see an increase. Similarly, service interruptions, such as snowfall, lead to a buildup of waste in the following days. The current study uses a compensated approach, keeping total demand constant and ensuring the feasibility of the routes. However, simulations with uncompensated local fluctuations or collective increases in demand would allow for more realistic testing of the effectiveness of A-VNS in maintaining route consistency. Such scenarios would highlight the limitations of the model and the trade-off between consistency and distance, showing the extent to which the algorithm can adapt routes without compromising the reference solution. In conclusion, introducing fluctuations closer to reality, both spatial and temporal, is crucial to evaluate the robustness and operational boundaries of the multi-objective approach implemented via A-VNS.

Another aspect that could be interesting as an object of study is the adoption of an inverse approach, using the reference primary solution as a starting point rather than the perturbed primary solution for multi-objective optimization (distance and coherence). In this way, the A-VNS meta-heuristic would start from a maximum consistency basis and focus on adapting routes to new demand conditions, minimizing costs without compromising similarity to the original solution. This approach is particularly useful in contexts where disturbances are temporary or time-limited, such as local events or road closures, as it allows for rapid restoration of operational routine without having to completely redesign routes, maximizing operational efficiency and drivers' familiarity with the routes.

References

[1] OECD. "Oecd glossary of statistical terms." (2008), [Online]. Available: https://www.oecd.org/content/dam/oecd/en/publications/reports/2008/09/oecd-glossary-of-statistical-terms_g1gh9ad7/9789264055087-en.pdf (visited on 11/11/2025).

[2] D. C. Wilson, "Learning from the past to plan for the future: An historical review of the evolution of waste and resource management 1970–2020 and reflections on priorities 2020–2030 – the perspective of an involved witness," *Waste Management & Research*, vol. 41, no. 12, pp. 1754–1813, 2023. DOI: 10.1177/0734242X231178025. [Online]. Available: https://doi.org/10.1177/0734242X231178025.

[3] W. Li, T. F. Ng, H. Ibrahim, and S. L. Wang, "A literature review of the state of the art of sustainable waste collection and vehicle routing problem," *Journal of the Air & Waste Management Association*, vol. 75, no. 1, pp. 3–26, 2025.

[4] W. Li, P. Wang, Y. Xu, L. Pan, C. Nie, and B. Yang, "Multi-objective optimization of municipal solid waste collection based on adaptive large neighborhood search," *Electronics*, vol. 14, no. 1, p. 103, 2024.

[5] H. Han and E. Ponce-Cueto, "Waste collection vehicle routing problem: A literature review," *PROMET - Traffic&Transportation*, vol. 27, Aug. 2015. DOI: 10.7307/ptt.v27i4.1616.

[6] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959. DOI: 10.1287/mnsc.6.1.80. [Online]. Available: https://doi.org/10.1287/mnsc.6.1.80.

[7] C. Hess, A. G. Dragomir, K. F. Doerner, and D. Vigo, "Waste collection routing: A survey on problems and methods," *Central European Journal of Operations Research*, 2024. DOI: 10.1007/s10100-023-00892-y. [Online]. Available: https://doi.org/10.1007/s10100-023-00892-y.

[8] A. Spinelli, F. Maggioni, T. R. P. Ramos, A. P. Barbosa-Póvoa, and D. Vigo, "A rolling horizon heuristic approach for a multi-stage stochastic waste collection problem," *European Journal of Operational Research*, vol. 323, no. 1, pp. 276–296, 2025.

[9] M. Dror, G. Laporte, and P. Trudeau, "Vehicle routing with stochastic demands: Properties and solution frameworks," *Transportation science*, vol. 23, no. 3, pp. 166–176, 1989.

[10] F. Firmansyah, I. Park, M. Corona, *et al.*, "Variation in municipal solid waste generation and management across time and space," *Resources, Conservation and Recycling*, vol. 204, p. 107 472, 2024.

[11] R. Šomplák, V. Smejkalová, M. Roseckỳ, *et al.*, "Comprehensive review on waste generation modeling," *Sustainability*, vol. 15, no. 4, p. 3278, 2023.

[12] G. Denafas, T. Ruzgas, D. Martuzevičius, *et al.*, "Seasonal variation of municipal solid waste generation and composition in four east european cities," *Resources, conservation and recycling*, vol. 89, pp. 22–30, 2014.

[13] S.-Y. Tan and W.-C. Yeh, "The vehicle routing problem: State-of-the-art classification and review," *Applied Sciences*, vol. 11, no. 21, 2021, ISSN: 2076-3417. [Online]. Available: https://www.mdpi.com/2076-3417/11/21/10295.

[14] E. J. Beltrami and L. D. Bodin, "Networks and vehicle routing for municipal waste collection," *Networks*, vol. 4, no. 1, pp. 65–94, 1974.

[15] O. A. C. Torres and F. Robusté, *A continuous approximation model for vehicle routing in solid waste management systems*. Universidad Politécnica de Catalunya, 2007.

[16] O. Apaydin and M. Gonullu, "Route optimization for solid waste collection: Trabzon (turkey) case study," *Global NEST Journal*, vol. 9, no. 1, pp. 6–11, 2007.

[17] M. Ghose, A. K. Dikshit, and S. Sharma, "A gis based transportation model for solid waste disposal–a case study on asansol municipality," *Waste management*, vol. 26, no. 11, pp. 1287–1293, 2006.

[18] A. Z. Alagöz and G. Kocasoy, "Improvement and modification of the routing system for the health-care waste collection and transportation in istanbul," *Waste management*, vol. 28, no. 8, pp. 1461–1471, 2008.

[19] H. Kim, J. Yang, and K.-D. Lee, "Vehicle routing in reverse logistics for recycling end-of-life consumer electronic goods in south korea," *Transportation Research Part D: Transport and Environment*, vol. 14, no. 5, pp. 291–299, 2009.

[20] R. M. Clark and J. I. Gillean, "Analysis of solid waste management operations in cleveland, ohio: A case study," *Interfaces*, vol. 6, no. 1-part-2, pp. 32–42, 1975.

[21] C. A. Arribas, C. A. Blazquez, and A. Lamas, "Urban solid waste collection system using mathematical modelling and tools of geographic information systems," *Waste Management & Research*, vol. 28, no. 4, pp. 355–363, 2010.

[22] N.-B. Chang and Y. Wei, "Siting recycling drop-off stations in urban area by genetic algorithm-based fuzzy multiobjective nonlinear integer programming modeling," *Fuzzy sets and systems*, vol. 114, no. 1, pp. 133–149, 2000.

[23] V. C. Hemmelmayr, K. F. Doerner, R. F. Hartl, and D. Vigo, "Models and algorithms for the integrated planning of bin allocation and vehicle routing in solid waste management," *Transportation Science*, vol. 48, no. 1, pp. 103–120, 2014.

[24] M. Cubillos and S. Wøhlk, "Solution of the maximal covering tour problem for locating recycling drop-off stations," *Journal of the Operational Research Society*, vol. 72, no. 8, pp. 1898–1913, 2021. DOI: 10.1080/01605682.2020.1746701.

[25] X. Bing, M. de Keizer, J. M. Bloemhof-Ruwaard, and J. G. van der Vorst, "Vehicle routing for the eco-efficient collection of household plastic waste," *Waste management*, vol. 34, no. 4, pp. 719–729, 2014.

[26] C. Xin, L. Wang, B. Liu, Y.-H. Yuan, and S.-B. Tsai, "[retracted] an empirical study for green transportation scheme of municipal solid waste based on complex data model analysis," *Mathematical Problems in Engineering*, vol. 2021, no. 1, p. 6 614 312, 2021.

[27] T. R. P. Ramos, M. I. Gomes, and A. P. Barbosa-Póvoa, "Economic and environmental concerns in planning recyclable waste collection systems," *Transportation Research Part E: Logistics and Transportation Review*, vol. 62, pp. 34–54, 2014.

[28] S. Coene, A. Arnout, and F. C. Spieksma, "On a periodic vehicle routing problem," *Journal of the Operational Research Society*, vol. 61, no. 12, pp. 1719–1728, 2010.

[29] P. Francis, K. Smilowitz, and M. Tzur, "The period vehicle routing problem with service choice," *Transportation science*, vol. 40, no. 4, pp. 439–454, 2006.

[30] H. Öztop, D. Kizilay, and Z. A. Çil, "Mathematical models for the periodic vehicle routing problem with time windows and time spread constraints," *An International Journal of Optimization and Control Theories & Applications (IJOCTA)*, vol. 11, pp. 10–23, Sep. 2020. DOI: 10.11121/ijocta.01.2021.00899.

[31] A. A. Kovacs, B. L. Golden, R. F. Hartl, and S. N. Parragh, "Vehicle routing problems in which consistency considerations are important: A survey," *Networks*, vol. 64, no. 3, pp. 192–213, 2014.

[32] I. Rodríguez-Martín and H. Yaman, "Periodic vehicle routing problem with driver consistency and service time optimization," *Transportation Research Part B: Methodological*, vol. 166, pp. 468–484, 2022.

[33] B. Bozkaya, F. S. Salman, and K. Telciler, "An adaptive and diversified vehicle routing approach to reducing the security risk of cash-in-transit operations," *Networks*, vol. 69, no. 3, pp. 256–269, 2017.

[34] Z. Xu and Y. Cai, "Variable neighborhood search for consistent vehicle routing problem," *Expert Systems with Applications*, vol. 113, pp. 66–76, 2018.

[35] J. Legriel, C. Le Guernic, S. Cotton, and O. Maler, "Approximating the pareto front of multi-criteria optimization problems," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, Springer, 2010, pp. 69–83.

[36] F. W. Glover and G. A. Kochenberger, *Handbook of metaheuristics*. Springer Science & Business Media, 2003, vol. 57.

[37] M. Gendreau, J.-Y. Potvin, *et al.*, *Handbook of metaheuristics*. Springer, 2010, vol. 2.

[38] M. Dorigo, A. Colorni, and V. Maniezzo, "Distributed optimization by ant colonies," in *Proceedings of the first European conference on artificial life*, 1991, pp. 134–142.

[39] J.-F. Cordeau, M. Gendreau, and G. Laporte, "A tabu search heuristic for periodic and multi-depot vehicle routing problems," *Networks: An International Journal*, vol. 30, no. 2, pp. 105–119, 1997.

[40] F. Alonso, M. J. Alvarez, and J. E. Beasley, "A tabu search algorithm for the periodic vehicle routing problem with multiple vehicle trips and accessibility restrictions," *Journal of the Operational Research Society*, vol. 59, no. 7, pp. 963–976, 2008.

[41] L. C. Coelho, J. Renaud, and G. Laporte, "Road-based goods transportation: A survey of real-world logistics applications from 2000 to 2015," *INFOR: Information Systems and Operational Research*, vol. 54, no. 2, pp. 79–96, 2016.

[42] D. Aksen, O. Kaya, F. S. Salman, and Ö. Tüncel, "An adaptive large neighborhood search algorithm for a selective and periodic inventory routing problem," *European Journal of Operational Research*, vol. 239, no. 2, pp. 413–426, 2014.

[43] C. Chen, E. Demir, and Y. Huang, "An adaptive large neighborhood search heuristic for the vehicle routing problem with time windows and delivery robots," *European journal of operational research*, vol. 294, no. 3, pp. 1164–1180, 2021.

[44] J. Kytöjoki, T. Nuortio, O. Bräysy, and M. Gendreau, "An efficient variable neighborhood search heuristic for very large scale vehicle routing problems," *Computers & operations research*, vol. 34, no. 9, pp. 2743–2757, 2007.

[45] V. Hemmelmayr, K. F. Doerner, R. F. Hartl, and S. Rath, "A heuristic solution method for node routing based solid waste collection problems," *Journal of Heuristics*, vol. 19, no. 2, pp. 129–156, 2013.

[46] N. A. Wouda, L. Lan, and W. Kool, "Pyvrp: A high-performance vrp solver package," *INFORMS Journal on Computing*, vol. 36, no. 4, pp. 943–955, 2024.

[47] S. Eilon, C. D. T. Watson-Gandy, N. Christofides, and R. de Neufville, "Distribution management-mathematical modelling and practical analysis," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 6, pp. 589–589, 1974.

[48] N. Christofides and J. E. Beasley, "The period routing problem," *Networks*, vol. 14, no. 2, pp. 237–256, 1984.

[49] R. Russell and W. Igo, "An assignment routing problem," *Networks*, vol. 9, no. 1, pp. 1–17, 1979.

[50] R. A. Russell and D. Gribbin, "A multiphase approach to the period routing problem," *Networks*, vol. 21, no. 7, pp. 747–765, 1991.

[51] I.-M. Chao, B. L. Golden, and E. A. Wasil, "A new heuristic for the period traveling salesman problem," *Computers & Operations Research*, vol. 22, no. 5, pp. 553–565, 1995.

[52] J.-F. Cordeau, M. Gendreau, and G. Laporte, *Pvrp old instances (p01–p32)*, Online, Available: `http://neumann.hec.ca/chairedistributique/data/pvrp/old/`, 1997.

[53] M. A. Haughton, "Assigning delivery routes to drivers under variable customer demands," *Transportation Research Part E: Logistics and Transportation Review*, vol. 43, no. 2, pp. 157–172, 2007.

[54] M. Gazda, *Tsp algorithms: 2-opt, 3-opt in python — matej gazda*, 2019. [Online]. Available: `http://matejgazda.com/tsp-algorithms-2-opt-3-opt-in-python/`.

[55] D. Gulczynski, B. Golden, and E. Wasil, "The period vehicle routing problem: New heuristics and real-world variants," *Transportation Research Part E: Logistics and Transportation Review*, vol. 47, no. 5, pp. 648–668, 2011.

[56] I.-M. Chao, B. L. Golden, and E. Wasil, "An improved heuristic for the period vehicle routing problem," *Networks*, vol. 26, no. 1, pp. 25–44, 1995.

[57] J. Alegre, M. Laguna, and J. Pacheco, "Optimizing the periodic pick-up of raw materials for a manufacturer of auto parts," *European Journal of Operational Research*, vol. 179, no. 3, pp. 736–746, 2007.

[58] V. C. Hemmelmayr, K. F. Doerner, and R. F. Hartl, "A variable neighborhood search heuristic for periodic routing problems," *European Journal of Operational Research*, vol. 195, no. 3, pp. 791–802, 2009.

Appendix A

Algorithms

Assignment Algorithm

```
def assignment_algorithm(
    k_vehicles, t_days, n_clients, max_clients_kt, vehicle_capacity,
        sorted_data, distance_matrix, distance_matrix_adjusted,
        closeness_matrix):

    '''
    'S' stands for supposition, meaning that it is assumed the customer
        is assigned. They are checked and then assigned only where chosen.
    '''

    # Create output solution
    (OBJ_tot_dist, assigned_ordered_matrix, not_assigned_list,
        transp_demand_matrix, route_dist_matrix) = output_matrices(
        k_vehicles, t_days, n_clients, max_clients_kt)

    for sor_client_i in range(1, len(sorted_data)):  # skip the depot
        # Assume to assign client to all the routes
        (real_client_index, S_assigned_cust_matrix) = assign_client(
        sor_client_i, k_vehicles, t_days, sorted_data,
            assigned_ordered_matrix)
        # Update vehicle capacities
        (demand_i, S_loads_matrix) = load_vehicle(
        sor_client_i, k_vehicles, t_days, sorted_data,
            transp_demand_matrix)
        # Verify vehicle maximmum capacity constraints
        (S_poss_assign_matrix) = check_capacity
```

```
20          (k_vehicles, t_days, vehicle_capacity, S_loads_matrix)
21          # Given the availabel routes, define possible schedules (visit
                combinations)
22          (S_visit_comb_matrix) = define_schedules(
23          sor_client_i, t_days, sorted_data)
24          # Calculate frequency of possible visit combinations and the
                required one
25          (min_freq_i, S_fesibility_matrix) = impose_schedules
26          (sor_client_i, sorted_data, S_poss_assign_matrix,
                S_visit_comb_matrix)
27
28          # If the required frequecy cannot be respected, add client to
                not_assigned_list
29          if not np.any(S_fesibility_matrix >= min_freq_i):
30              first_zero = np.where(not_assigned_list == 0)[0][0]
31              not_assigned_list[first_zero] = real_client_index
32              continue     # go to the next client
33
34          # Remove client from unused routes
35          (S_fesibility_matrix, S_visit_comb_matrix, S_poss_assign_matrix)
                = update_matrices(
36          min_freq_i, S_fesibility_matrix, S_visit_comb_matrix,
                S_poss_assign_matrix)
37          (S_fesibility_matrix) = update_feasibility(
38          min_freq_i, S_fesibility_matrix)
39
40          # Calculate route distances and schedule distances
41          (S_distances_matrix, S_assigned_cust_matrix) =
                calculate_closness_route_distances(
42          k_vehicles, t_days, S_poss_assign_matrix,  S_assigned_cust_matrix
                , distance_matrix_adjusted, closeness_matrix)
43          (S_sched_dist_matrix) = calculate_sched_dist(
44          S_distances_matrix, S_visit_comb_matrix, S_fesibility_matrix)
45          # Choose the schedule that guarantees lower distance
46          (winning_vehicle, winning_schedule) = find_winning_comb(
47          S_sched_dist_matrix)
48          # Update uotput matrices
49          (assigned_ordered_matrix, transp_demand_matrix, route_dist_matrix
                ) = update_out_matrices(
50              k_vehicles, t_days, winning_vehicle, winning_schedule,
51              assigned_ordered_matrix, transp_demand_matrix,
                    route_dist_matrix,
52              S_visit_comb_matrix, S_assigned_cust_matrix, S_loads_matrix,
```

```
            S_distances_matrix)
53
54      # Add depot befor and after each route
55      (assigned_ordered_matrix) = add_depot(k_vehicles, t_days,
            assigned_ordered_matrix)
56
57      # Calculate final route distances
58      for vehicle_k in range(k_vehicles):
59          for day_t in range(t_days):
60              route = copy.copy(assigned_ordered_matrix[vehicle_k, day_t])
61              route_distance = sum(distance_matrix[route[i], route[i + 1]]
                    for i in range(len(route) - 1))
62              route_dist_matrix[vehicle_k, day_t] = route_distance
63
64      # Calculate total distance (OBJ)
65      (OBJ_tot_dist) = calculate_tot_dist(k_vehicles, t_days,
            route_dist_matrix, OBJ_tot_dist)
66
67      # Save solution in class "Solution"
68      initial_solution = classes.Solution(OBJ_tot_dist,
            assigned_ordered_matrix, not_assigned_list, transp_demand_matrix,
            route_dist_matrix)
69
70      return initial_solution
```

A-VNS Algorithm

```
def AVNS_algorithm(
        n_vehicles, n_days, n_clients, max_clients_kt, vehicle_capacity,
        data, sorted_data, distance_matrix, distance_matrix_adjusted,
        closeness_matrix,
        initial_solution, current_solution, best_solution,
        time_limit_AVNS, max_no_improve_time,
        neighborhoods, max_neighbor, max_iteration_neighbor,
        ws_counter_limit, worse_sol_percentage,
        initial_score, rewards_values,
        instance_number,
        sim_type, weights):

    ''' Prepare parameters '''
    neighbour = 0
    iteration = 0
    ws_counter = 0
    score_neigh = np.zeros((max_neighbor+1), dtype=float)
    probability_neigh = np.zeros((max_neighbor+1), dtype=float)

    ''' START ALGORITHM '''
    start_time_VND = time.time()
    last_improvement_time = time.time()

    while time.time() - start_time_VND < time_limit_AVNS:

        # If time exceed limit time without improvent
        if (time.time() - last_improvement_time) > max_no_improve_time:
            break

        # Choode neighborhood and reset parameters
        neighbour = choose_neigh(probability_neigh, neighborhoods)
        iteration = 0
        ws_counter = 0
        total_run = 0
        reward_points = 0

        while iteration < max_iteration_neighbor:

            # Find the new solution (NS) based on the picked neighborhood
            (initial_comb, final_comb, new_solution, error_index) =
                define_neighboring_solution(
```

```
41                  sim_type, weights, neighbour,
42                  n_vehicles, n_days, vehicle_capacity, data,
43                  sorted_data, distance_matrix, closeness_matrix,
44                  initial_solution, current_solution, neighborhoods)
45
46          # If the neighbor. is not able to find a new solution, move
                to next iteration
47          if error_index == False:
48              reward_points += rewards_values[3]
49              iteration += 1
50              ws_counter += 1
51              total_run += 1
52              continue
53
54          # Verify solution feasibility and upsate soluion
                feasibility_vector
55          new_solution = feasibility_function(
56                  n_vehicles, n_days, n_clients, data, max_clients_kt,
57                  vehicle_capacity, distance_matrix, new_solution)
58          # If new solution is feasible:
59          if new_solution.feasibility_vector.verify() == True:
60
61              # NS is the new global best solution
62              if new_solution.OBJ_value < best_solution.OBJ_value:
63                  best_solution = copy.deepcopy(new_solution)
64                  current_solution = copy.deepcopy(new_solution)
65                  reward_points += rewards_values[0]
66                  iteration = max_iteration_neighbor + 1
67                  last_improvement_time = time.time()
68
69              # NS is better that current solution
70              elif new_solution.OBJ_value < current_solution.OBJ_value:
71                  current_solution = copy.deepcopy(new_solution)
72                  reward_points += rewards_values[1]
73                  iteration += 1
74
75              # NS is accepted as worsening solution
76              elif ws_counter == ws_counter_limit:
77                  OBJ_limit_value = (current_solution.OBJ_value * (1 +
                        worse_sol_percentage))
78                  if new_solution.OBJ_value < OBJ_limit_value:
79                      current_solution = copy.deepcopy(new_solution)
80                      reward_points += rewards_values[2]
```

```
81                          iteration += 1
82                          ws_counter = 0
83
84                  # NS is rejected
85                  else:
86                      reward_points += rewards_values[3]
87                      iteration += 1
88                      ws_counter += 1
89
90              # NS is unfeasible
91              else:
92                  reward_points += rewards_values[3]
93                  iteration += 1
94                  ws_counter += 1
95
96          # Update neighborhood scores and probabilities
97          actual_score = copy.copy(score_neigh[neighbour])
98          score_neigh[neighbour] = max(actual_score + ((reward_points -
                actual_score) / total_run), 0.001)
99          total_scores = sum(score_neigh)
100         for neigh in range(len(probability_neigh)):
101             probability_neigh[neigh] = score_neigh[neigh] / total_scores
102
103         # Optimize customer order in routes with 2-3-opt
104         current_solution = two_three_opt(sim_type, weights, n_vehicles,
                n_days, n_clients, max_clients_kt, distance_matrix,
                initial_solution, current_solution)
105
106     return (best_solution)
```

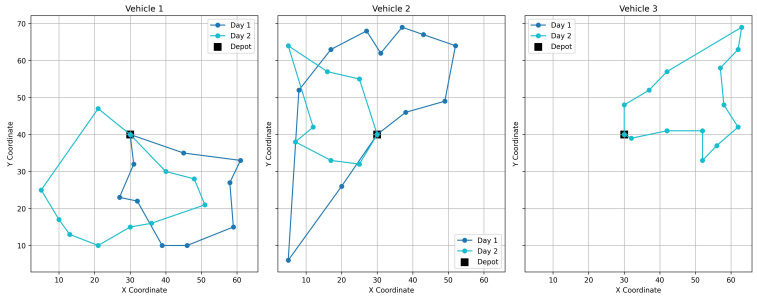
Appendix B

Table 1: Vehicle couplings and similarity measures of instance *p01*.

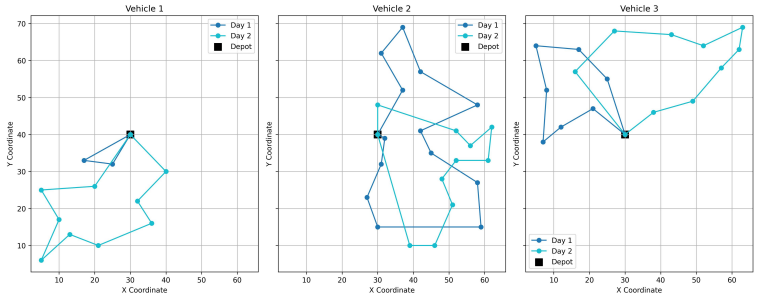
		sim_1		sim_2		sim_3	
		couplings	values	couplings	values	couplings	values
Primary refined solution (100 − 0)		(1,1)	0,3684	(1,2)	0,0417	(1,2)	0,0455
		(2,3)	0,6667	(2,3)	0,0000	(2,3)	0,0000
		(3,2)	0,7143	(3,1)	0,1667	(3,1)	0,2500
75-25	**sim_1**	(1,3)	0,8421	(1,3)	0,0417	(1,3)	0,0909
		(2,2)	0,7778	(2,2)	0,0000	(2,2)	0,0000
		(3,1)	0,7143	(3,1)	0,3667	(3,1)	0,4643
	sim_2	(1,1)	0,2632	(1,2)	0,0455	(1,2)	0,1000
		(2,3)	0,4444	(2,3)	0,0769	(2,3)	0,0833
		(3,2)	0,5714	(3,1)	0,2000	(3,1)	0,2857
	sim_3	(1,3)	0,5790	(1,2)	0,1250	(1,2)	0,1818
		(2,2)	0,4444	(2,1)	0,1325	(2,1)	0,1458
		(3,1)	0,5000	(3,3)	0,1667	(3,3)	0,2500
50-50	**sim_1**	(1,1)	0,9474	(1,1)	0,0455	(1,1)	0,1000
		(2,3)	0,8889	(2,3)	0,0556	(2,3)	0,0625
		(3,2)	0,9286	(3,2)	0,1667	(3,2)	0,2500
	sim_2	(1,1)	0,4211	(1,3)	0,2273	(1,3)	0,2500
		(2,3)	0,3333	(2,1)	0,2222	(2,1)	0,2500
		(3,2)	0,5714	(3,2)	0,3000	(3,2)	0,4286
	sim_3	(1,2)	0,6842	(1,2)	0,1742	(1,2)	0,3818
		(2,3)	0,7222	(2,3)	0,2650	(2,3)	0,3542
		(3,1)	0,7143	(3,1)	0,3000	(3,1)	0,3929

Continues on the next page

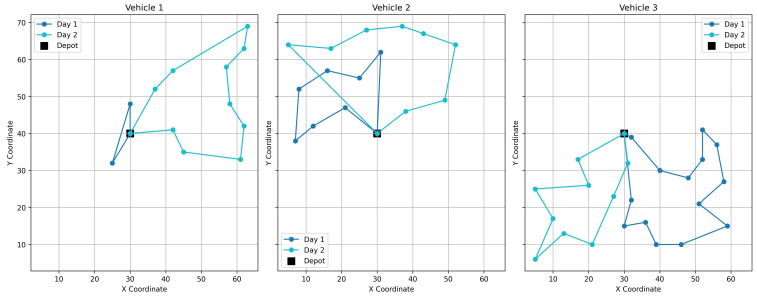
		sim_1		sim_2		sim_3	
		couplings	values	couplings	values	couplings	values
25-75	**sim_1**	(1,1)	0,8947	(1,1)	0,0000	(1,1)	0,0000
		(2,3)	0,8889	(2,3)	0,0000	(2,3)	0,0000
		(3,2)	0,9286	(3,2)	0,4333	(3,2)	0,5357
	sim_2	(1,2)	0,7368	(1,2)	0,3902	(1,2)	0,4273
		(2,3)	0,5000	(2,3)	0,3376	(2,3)	0,3750
		(3,1)	0,5714	(3,1)	0,3667	(3,1)	0,4643
	sim_3	(1,3)	0,7368	(1,3)	0,4773	(1,3)	0,5682
		(2,2)	0,7222	(2,2)	0,4915	(2,2)	0,5417
		(3,1)	0,6429	(3,1)	0,3667	(3,1)	0,5000
0-100	**sim_1**	(1,2)	0,8947	(1,1)	0,0000	(1,1)	0,0000
		(2,3)	0,8889	(2,3)	0,0556	(2,3)	0,0625
		(3,1)	0,9286	(3,2)	0,1667	(3,2)	0,2500
	sim_2	(1,2)	0,7368	(1,2)	0,3939	(1,2)	0,4318
		(2,1)	0,7222	(2,1)	0,4316	(2,1)	0,4792
		(3,3)	0,7857	(3,3)	0,3667	(3,3)	0,4643
	sim_3	(1,2)	0,7895	(1,2)	0,5682	(1,2)	0,6227
		(2,1)	0,7222	(2,1)	0,5470	(2,1)	0,6042
		(3,3)	0,8571	(3,3)	0,4000	(3,3)	0,6071



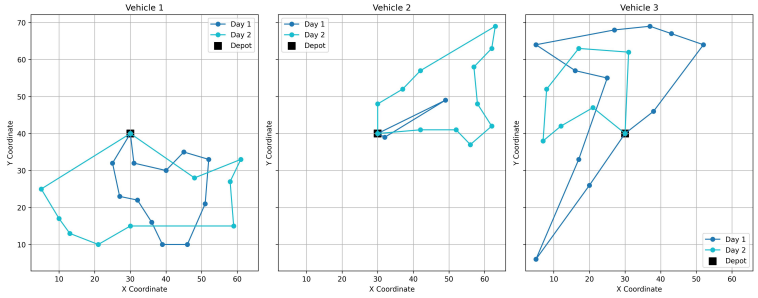
(a) Primary reference solution.



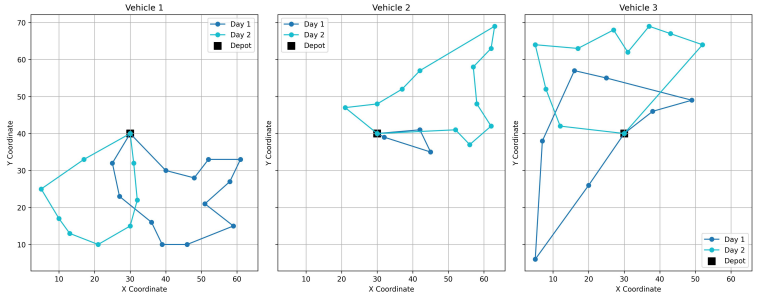
(b) Primary perturbed solution, (100, 0).



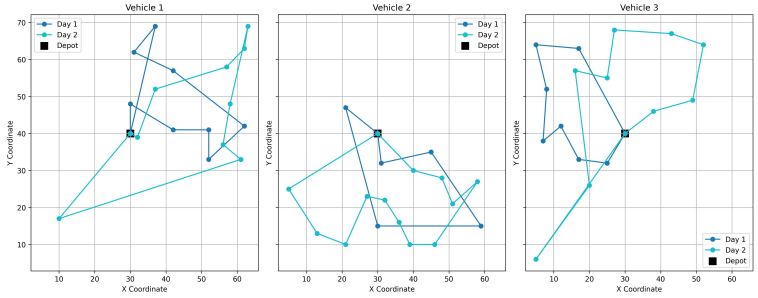
(c) Refined perturbed solution of simulation (75, 25).



(d) Refined perturbed solution of simulation (50, 50).

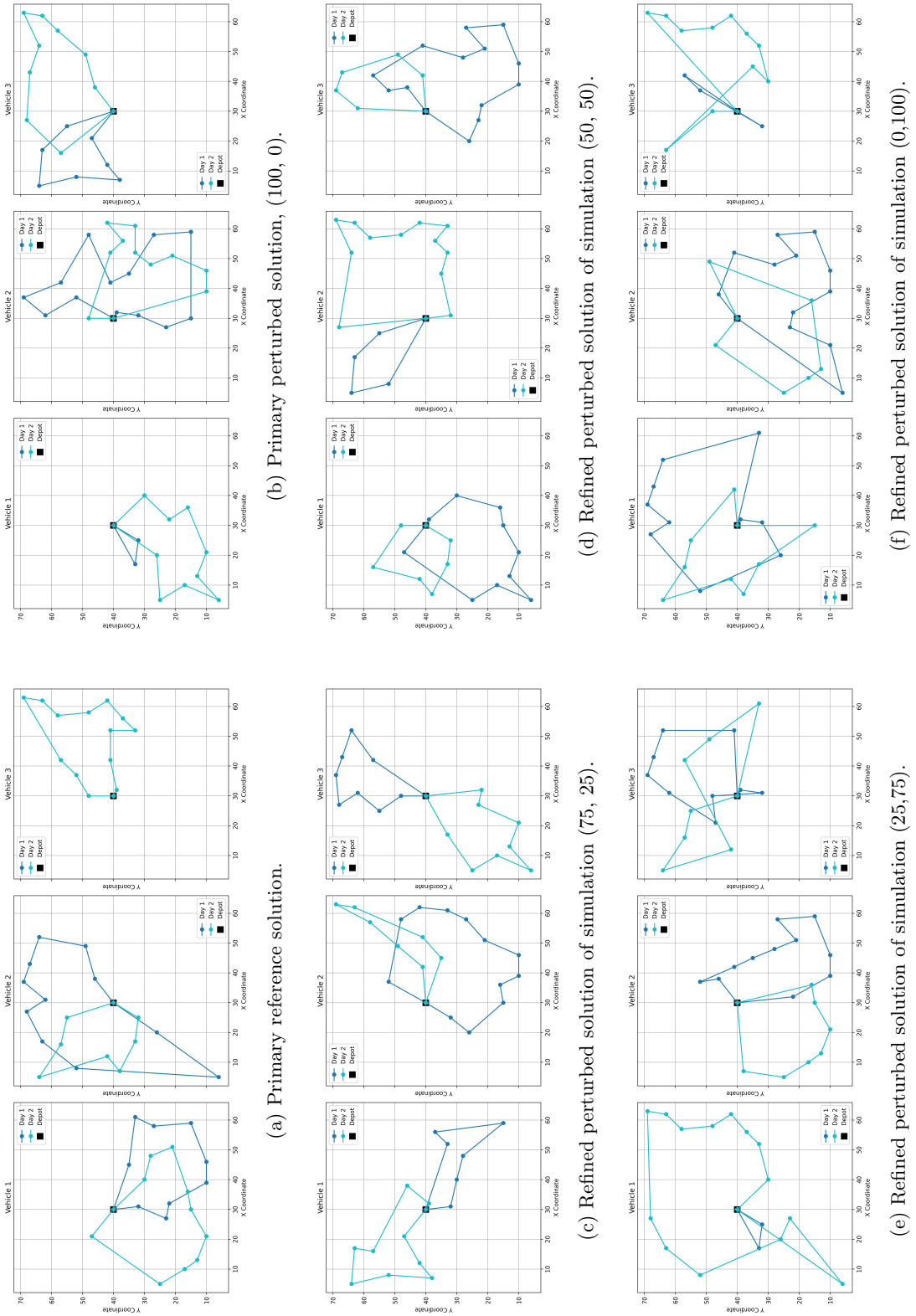


(e) Refined perturbed solution of simulation (25,75).



(f) Refined perturbed solution of simulation (0,100).

Figure 1: Routes plots in the different simulations optimizing *sim_1*.



(a) Primary reference solution.

(b) Primary perturbed solution, (100, 0).

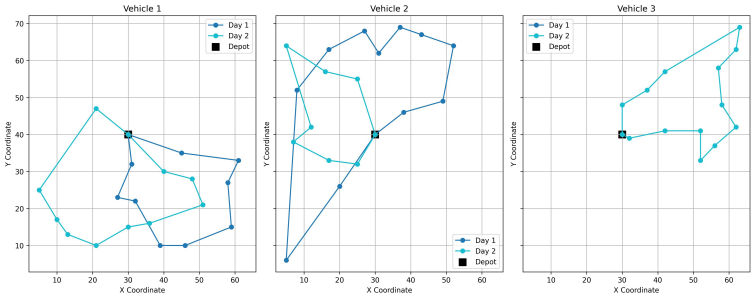
(c) Refined perturbed solution of simulation (75, 25).

(d) Refined perturbed solution of simulation (50, 50).

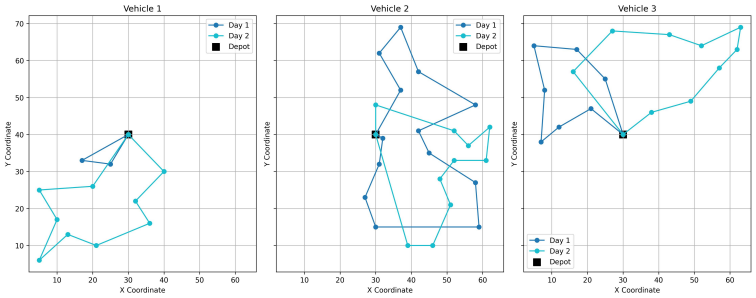
(e) Refined perturbed solution of simulation (25,75).

(f) Refined perturbed solution of simulation (0,100).

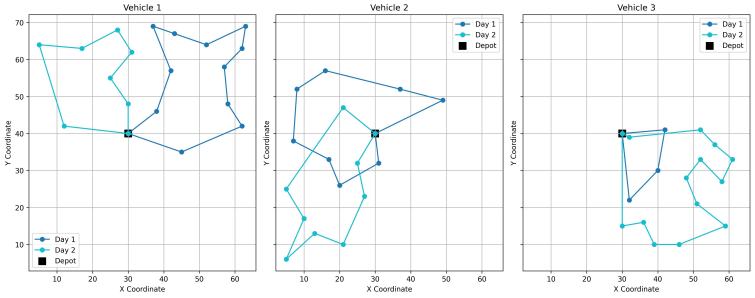
Figure 2: Routes plots in the different simulations optimizing *sim_2*.



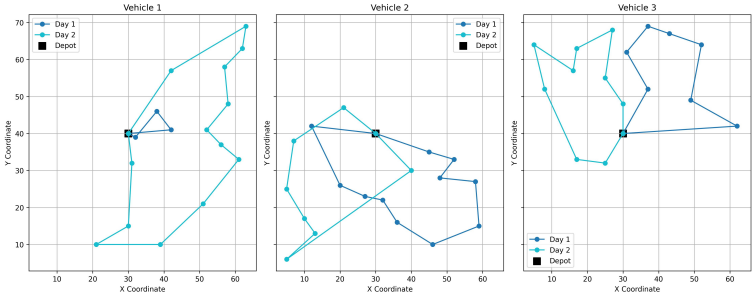
(a) Primary reference solution.



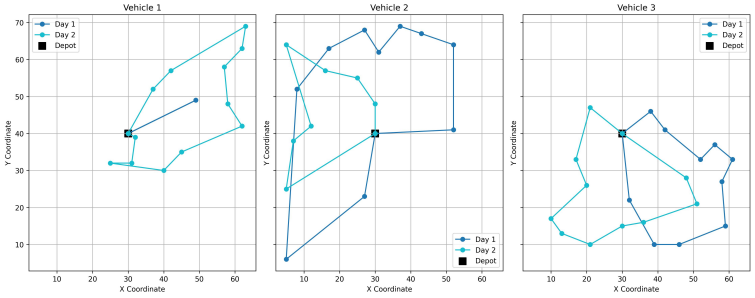
(b) Primary perturbed solution, (100, 0).



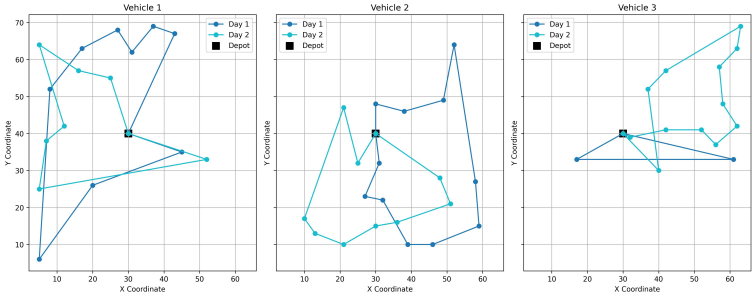
(c) Refined perturbed solution of simulation (75, 25).



(d) Refined perturbed solution of simulation (50, 50).



(e) Refined perturbed solution of simulation (25,75).



(f) Refined perturbed solution of simulation (0,100).

Figure 3: Routes plots in the different simulations optimizing *sim_3*.